

PC FORGE

Ghidul final de invatat pentru prezentare

Etapele 5, 6 si 7 - cod, demonstratie si explicatii verificate

Obiectiv: Sa poti arata functionalitatea, sa urmaresti datele pana in cod si sa raspunzi natural la intrebarea «de ce ai implementat asa?». Ghidul nu cere memorarea codului caracter cu caracter.

Popa Bogdan | PC Forge | material de studiu si demonstratie

Cum inveti eficient din acest ghid

Ghidul separa cerintele noi de efectele deja prezentate, ca sa nu numeri aceleasi puncte de doua ori. Din pachetul extins ai nevoie de aproximativ 4,98 puncte pentru nota 10. Bonusurile 1, 13 si 17 din Etapa 6 sunt capitole de rezerva: invata-le dupa ce poti prezenta fluent pachetul principal. Fiecare capitol iti spune exact ce fila deschizi, ce apesi si ce explici.

Formula universală de raspuns: Cerinta cere X. Datele pornesc din Y. Functia Z le transforma astfel. Rezultatul este W, pe care il demonstrez in browser. Am ales aceasta solutie deoarece respecta cerinta si separa responsabilitatile.

Pregatirea calculatorului

Porneste Docker Desktop si asteapta ca PostgreSQL PC Forge sa fie disponibil.

Deschide proiectul C:\Users\bogdi\Desktop\PC-Forge in VS Code si porneste serverul cu npm start.

Deschide <http://localhost:8080>, <http://localhost:8080/produse> si o fereastră privata pentru bannerul cookie.

In VS Code foloseste cautarea dupa numele functiei. Nu derula la intamplare prin index.js.

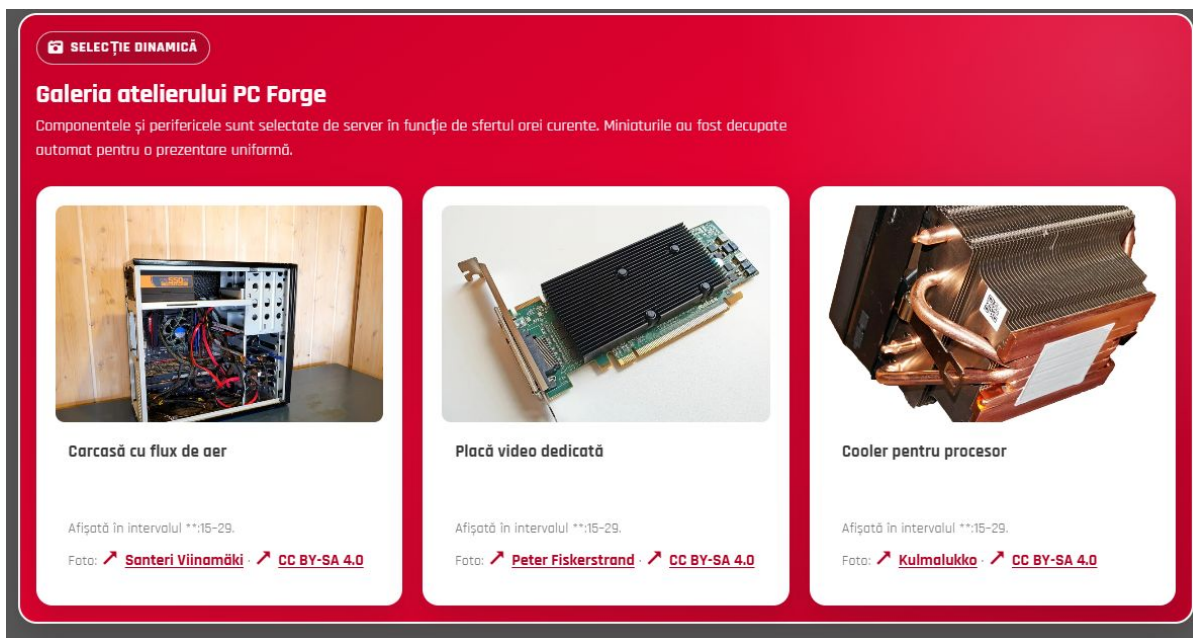
Nu afisa fisierul .env, parole, date SMTP sau hashuri reale in fata clasei.

Ordinea prezentarii

Incepi cu rezultatul vizibil. Dupa ce profesoara vede ca functioneaza, treci in VS Code si arati numai functia relevanta. Revii apoi in browser. Aceasta ordine dovedeste functionalitatea si evita pierderea timpului in cod.

ETAPA 5 - pachet nou selectat: 1,475 puncte

Ideea etapei: Galeria si Bootstrap-ul arata rezultatul, iar serverul automatizeaza selectarea imaginilor, compilarea Sass, backupul si validarea resurselor.



Galeria statica PC Forge - rezultatul serverului si al gridului responsive.

5.1 Galeria statica (0,35 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#galerie-statica> | VS Code: `resurse/json/galerie.json`; `index.js`; `views/fragmente/galerie-statica.ejs`; `resurse/sass/galerie.scss`

Ce cere exact: Imaginile trebuie citite din JSON, filtrate pe server dupa sfertul orei, afisate cu figure si figcaption, dimensionate uniform si organizate responsive in 3, 2 sau 1 coloana.

Demonstratia, in ordinea corecta

Deschide galeria si indica imaginile de aceeasi dimensiune si descrierile lor.

Redimensioneaza fereastra: trei coloane pe ecran mare, doua pe mediu si una pe mic.

Treci cu mouse-ul peste o imagine si arata transformarea, rotirea si modificarea chenarului.

In VS Code arata obiectul JSON, filtrarea pe server si bucla EJS.

Cum functioneaza in proiect

`galerie.json` este sursa de date. Fiecare obiect contine calea imaginii, titlul, descrierea si `sfert_ora`.

`obtainImaginiGalerieStatica` ruleaza pe server. Minutele sunt impartite in patru intervale: 1 inseamna 00-14, 2 inseamna 15-29, 3 inseamna 30-44, iar 4 inseamna 45-59. Nicio valoare nu inseamna imagine permanenta.

Serverul transmite template-ului numai imaginile eligibile. EJS nu primeste toate imaginile pentru a le ascunde ulterior.

figure grupeaza semantic imaginea cu `figcaption`, CSS Grid controleaza coloanele, iar tranzitia de hover dureaza exact o secunda, conform cerintei individuale.

Algoritmul pe care trebuie sa il intelegi

Citeste minutul curent.

Calculeaza sfertul orei prin `Math.floor(minute / 15) + 1`.

Pastreaza numai imaginile al caror `sfert_ora` este egal cu sfertul curent.

Limiteaza rezultatul la maximum zece imagini.

EJS parcurge vectorul cu `forEach` si produce markupul.

Media queries schimba numarul coloanelor; tranzitia de o secunda anima proprietatile de hover.

Bucata-cheie: functia care face filtrarea pe server:

```
510 | function obtineImaginiGalerieStatica() {
511 |   const sfertOraCurent = Math.floor(new Date().getMinutes() / 15) + 1;
512 |
513 |   return obGlobal.obGalerie.imagini
514 |     .filter(function (image) {
515 |       return image.sfert_ora === sfertOraCurent && image.cale_thumbnail;
516 |     })
517 |     .slice(0, 10);
518 | }
519 |
520 | /**
521 |  * Alege aleator un grup consecutiv de imagini pentru galeria animata.
522 |  * @returns {{imagini: object[], numarImagini: number}} Datele galeriei animate.
```

Intrebari probabile si raspunsuri

De ce nu ascunzi imaginile cu CSS?

Raspuns: Cerinta spune ca serverul trebuie sa trimita numai imaginile relevante. CSS ar primi oricum toate datele.

De ce figure si nu doar div?

Raspuns: figure si figcaption exprima semantic faptul ca legenda apartine imaginii.

Ce inseamna responsive?

Raspuns: Componentele se reorganizeaza dupa spatiul disponibil; nu este doar micșorarea întregii pagini.

5.2 Compilarea automata SCSS (0,25 p)

Ce trebuie sa deschizi: Browser: Terminalul VS Code si orice pagina a site-ului | VS Code: index.js; resurse/sass; resurse/css; backup/resurse/css

Ce cere exact: Serverul defineste folderele SCSS/CSS/backup, compileaza toate sursele la pornire, salveaza CSS-ul anterior si recompila automat fisierul modificat.

Demonstratia, in ordinea corecta

Arata mesajele de compilare din terminal la pornire.

Modifica o valoare nepericuloasa dintr-un SCSS si salveaza.

Arata aparitia mesajului de recompilare si apoi schimbarea in browser.

Deschide backup/resurse/css si arata copia precedenta.

Cum functioneaza in proiect

SCSS este sursa pentru dezvoltare; browserul citeste CSS-ul rezultat.

compileazaScss rezolva caile absolute sau relative, creeaza folderul destinatie, face backup si apeleaza sass.compile.

compileazaToateScss parcurge sursele la pornire. Fisierul partial care incepe cu underscore sunt importate de alte fisier si nu trebuie compilate separat.

urmarestScss foloseste fs.watch. Debounce-ul evita mai multe compilari pentru aceeasi salvare.

Algoritmul pe care trebuie sa il intelegi

Determina calea sursa si destinatia.

Daca exista CSS vechi, il copiaza in backup.

Compileaza SCSS cu pachetul sass.

Scrie CSS-ul rezultat.

Watcherul observa modificarile si reapele functia dupa o intarziere scurta.

Bucata-cheie: inceputul functiei compileazaScss si fluxul backup-compilare:

```

252 | function compileazaScss(caleScss, caleCss) {
253 |   let caleScssAbs;
254 |   let caleCssAbs;
255 |
256 |   // Daca fisierul SCSS are cale absoluta, o folosim direct.
257 |   // Daca are cale relativa, o consideram relativa la folderScss.
258 |   if (path.isAbsolute(caleScss)) {
259 |     caleScssAbs = caleScss;
260 |   } else {
261 |     caleScssAbs = path.join(obGlobal.folderScss, caleScss);
262 |   }
263 |
264 |   // Daca s-a transmis calea CSS, o folosim.
265 |   // Altfel, generam automat numele CSS pornind de la numele SCSS.
266 |   if (caleCss) {
267 |     if (path.isAbsolute(caleCss)) {
268 |       caleCssAbs = caleCss;
269 |     } else {
270 |       caleCssAbs = path.join(obGlobal.folderCss, caleCss);
271 |     }
272 |   } else {
273 |     const caleRelativaScss = path.relative(obGlobal.folderScss, caleScssAbs);
274 |     const caleRelativaCss = caleRelativaScss.replace(/\.scss$/i, ".css");
275 |     caleCssAbs = path.join(obGlobal.folderCss, caleRelativaCss);
276 |   }
277 |
278 |   // Cream folderul destinatie pentru CSS, daca nu exista.
279 |   creeazaFolderDacaNuExista(path.dirname(caleCssAbs));
280 |
281 |   // Inainte de suprascriere, salvam CSS-ul vechi in backup.
282 |   if (fs.existsSync(caleCssAbs)) {
283 |     try {
284 |       const caleRelativaCss = path.relative(obGlobal.folderCss, caleCssAbs);
285 |       const infoFisierCss = path.parse(caleRelativaCss);
286 |
287 |       const numeBackup =
288 |         infoFisierCss.name + "_" + Date.now() + infoFisierCss.ext;
289 |
290 |       const caleBackup = path.join(
291 |         obGlobal.folderBackup,
292 |         "resurse",
293 |         "css",
294 |         infoFisierCss.dir,
295 |         numeBackup,
296 |       );
297 |
298 |       creeazaFolderDacaNuExista(path.dirname(caleBackup));
299 |       fs.copyFileSync(caleCssAbs, caleBackup);
300 |
301 |       console.log("Backup CSS creat: " + caleBackup);
302 |     } catch (eroareBackup) {
303 |       console.error("Eroare la crearea backupului pentru: " + caleCssAbs);
304 |       console.error(eroareBackup.message);
305 |     }

```

```

306 | }
307 |
308 | // Compilarea efectiva SCSS -> CSS
309 | try {
310 |   const rezultat = sass.compile(caleScssAbs, {
311 |     style: "expanded",
312 |     quietDeps: true,
313 |     // Bootstrap 5 foloseste inca API-uri Sass marcate pentru o versiune
314 |     // viitoare; avertismentele sunt ascunse, fara a ascunde
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce browserul nu incarca SCSS?

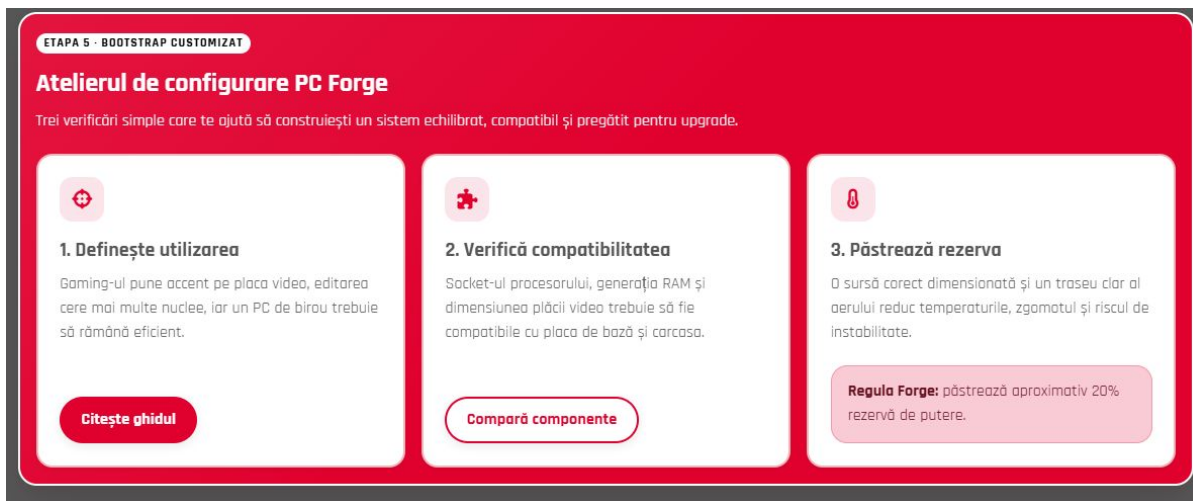
Raspuns: Browserele inteleg CSS, nu sintaxa Sass. Sass este compilat inainte.

Ce este debounce?

Raspuns: O intarziere care combina evenimente rapide intr-o singura compilare.

Ce se intampla daca Sass este invalid?

Raspuns: Compilatorul raporteaza eroarea; backupul pastreaza versiunea CSS anterioara.



Bootstrap customizat in schema PC Forge - componente reale, nu continut placeholder.

5.3 Bootstrap customizat prin Sass (0,25 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#bootstrap-custom> | VS Code: [resurse/sass/custom.scss](#); [resurse/css/custom.css](#); [views/pagini/index.ejs](#); [views/fragmente/head.ejs](#)

Ce cere exact: Bootstrap trebuie compilat cu variabile schimbate pentru culori, font, breakpointuri, raze, headinguri si alte componente, apoi folosit efectiv in pagina.

Demonstratia, in ordinea corecta

Arata cardurile PC Forge, butonul plin, butonul outline si alerta.

Redimensioneaza pagina pentru a demonstra gridul Bootstrap.

In SCSS indica variabilele declarate inaintea importului Bootstrap.

In head arata ca Bootstrap este incarcat inaintea CSS-ului proiectului.

Cum functioneaza in proiect

Variabilele Sass sunt evaluate la compilare. De aceea valorile PC Forge trebuie definite inainte de importul Bootstrap.

primary este rosul proiectului, secondary este griul, fontul este Rajdhani, breakpointurile si border-radius sunt personalizate.

Clasele row, col, card, btn si alert demonstreaza componente reale Bootstrap. Continutul explica tema PC Forge, nu este un placeholder tehnic.

CSS-ul Bootstrap este incarcat primul, iar stilurile locale vin dupa el pentru corectii precise.

Algoritmul pe care trebuie sa il intelegi

Suprascrie variabilele Sass.

Importa sursa Bootstrap.

Compilerul genereaza clasele cu noile valori.

EJS aplica acele clase componentelor.

CSS-ul local repara numai conflictele cu stilurile vechi.

Bucata-cheie: fisierul folosit efectiv pentru tema Bootstrap:

```

1 | /* Etapa 5 - tema Bootstrap PC Forge.
2 | Variabilele sunt suprascrise inainte de importul frameworkului. */
3 | $primary: #e1012f;
4 | $secondary: #575757;
5 | $danger: #e01816;
6 | $light: #ffffff;
7 | $dark: #1f1f1f;
8 | $body-bg: #575757;
9 | $body-color: #ffffff;
10 | $font-family-base: "Rajdhani", Arial, sans-serif;
11 | $h1-font-size: 3rem;
12 | $h2-font-size: 2.2rem;
13 | $h3-font-size: 1.6rem;
14 | $border-radius: 0.8rem;
15 | $border-radius-sm: 0.5rem;
16 | $border-radius-lg: 1.2rem;
17 | $border-radius-xl: 1.5rem;
18 | $grid-breakpoints: (xs: 0, sm: 576px, md: 760px, lg: 1100px, xl: 1320px, xxl: 1500px);
19 | $container-max-widths: (sm: 540px, md: 720px, lg: 1040px, xl: 1240px, xxl: 1440px);
20 | $btn-border-radius: 999px;
21 | $btn-border-width: 2px;
22 | $btn-padding-x: 1.2rem;
23 | $btn-padding-y: 0.55rem;
24 | $input-border-width: 2px;
25 | $input-border-radius: 0.8rem;
26 | $input-focus-border-color: #e1012f;
27 | $form-range-thumb-width: 1.5rem;
28 | $form-range-thumb-height: 1.5rem;
29 | $form-range-thumb-bg: #e1012f;
30 | $form-range-track-bg: #f5a3b4;
31 | $card-border-radius: 1rem;
32 | $card-border-color: rgba(#e02f35, 0.35);
33 | $card-bg: #ffffff;
34 | $card-color: #575757;
35 | $alert-border-radius: 1rem;
36 | $alert-padding-x: 1rem;
37 | $alert-padding-y: 0.8rem;
38 | @import "../node_modules/bootstrap/scss/bootstrap";

```

Intrebari probabile si raspunsuri

De ce primul buton este plin si al doilea outline?

Raspuns: Sunt doua stari Bootstrap intentionate: btn-primary este actiunea principala, iar btn-outline-primary este una secundara. Hoverul completeaza fundalul celui outline.

De ce variabilele sunt inaintea importului?

Raspuns: Bootstrap isi genereaza regulile folosind valorile existente in momentul importului.

Bootstrap inlocuieste CSS-ul proiectului?

Raspuns: Nu. Bootstrap ofera componente si utilitare; CSS-ul PC Forge pastreaza identitatea si rezolva conflictele locale.

5.4 Galeria animata (0,50 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/#galerie-animata` | VS Code: `index.js`; `views/fragmente/galerie-animata.ejs`; `resurse/sass/galerie.scss`

Ce cere exact: La fiecare incarcare serverul alege aleator un numar de imagini divizibil cu 3 si mai mic de 16, apoi o secventa consecutiva si distincta. Sass genereaza animatia, iar hover o pune pe pauza.

Demonstratia, in ordinea corecta

Reincarca pagina si observa ca selectia se poate schimba.

Urmareste ciclul animatiei si faptul ca ultima imagine continua natural spre prima.

Tine cursorul peste galerie si verifica pauza.

Micsoreaza fereastra si arata ca galerie dispare pe ecranele unde cerinta o solicita.

Cum functioneaza in proiect

Aleatoriul nu este complet liber: lista valorilor permise contine numai multipli de 3 sub 16 si care incap in JSON.

Offsetul poate fi orice indice al vectorului. Expresia $(\text{offset} + \text{index}) \% \text{lungime}$ formeaza o secventa circulara: dupa ultima imagine continua de la inceput.

EJS introduce numarul in clasa `galerie-animata--3`, `--6`, `--9` sau `--12`. Sass genereaza clasele si keyframe-urile pentru acele patru variante.

`animation-play-state: paused` opreste timpul animatiei pe hover, nu reseteaza pozitia.

Algoritmul pe care trebuie sa il intelegi

Construieste numerele eligibile.

Alege aleator numarul de imagini si offsetul.

Parcurge numarul cerut de pozitii.

Foloseste modulo pentru revenirea circulara la inceputul vectorului.

EJS creeaza elementele si clasa variantei.

Sass aplica animatia; hover comuta starea pe `paused`.

Bucata-cheie: selectia aleatoare circulara:

```

524 | function obtineGalerieAnimata() {
525 |   const imaginiDisponibile = obGlobal.obGalerie.imagini.filter(function (image) {
526 |     return image.cale_thumbnail;
527 |   });
528 |   const numerePermise = [3, 6, 9, 12].filter(function (numar) {
529 |     return numar < 16 && numar <= imaginiDisponibile.length;
530 |   });
531 |   const numarImagini =
532 |     numerePermise[Math.floor(Math.random() * numerePermise.length)];
533 |   const offset = Math.floor(Math.random() * imaginiDisponibile.length);
534 |   const imagini = [];
535 |
536 |   for (let index = 0; index < numarImagini; index++) {
537 |     imagini.push(imaginiDisponibile[(offset + index) % imaginiDisponibile.length]);
538 |   }
539 |
540 |   return { imagini: imagini, numarImagini: numarImagini };
541 | }

```

Intrebari probabile si raspunsuri

De ce numarul trebuie divizibil cu 3?

Raspuns: Este o constrangere explicita a cerintei si permite impartirea coerenta a secventei animate.

Cum garantezi imagini distincte?

Raspuns: Numarul ales nu depaseste lungimea vectorului, iar fiecare pas foloseste urmatorul indice; modulo doar continua de la inceput, fara sa repete un indice in acelasi ciclu.

Ce face operatorul %?

Raspuns: Returneaza restul impartirii si mentine indicele intre 0 si lungime minus 1.

De ce generezi CSS cu Sass?

Raspuns: Numarul imaginilor variaza, iar @for produce automat cadrele corespunzatoare.

Efectele CSS - ce este deja punctat si ce este nou

Nu dubla punctajul: HR-ul, duotone-ul, textul pe coloane si textul animat au fost deja prezentate si au adus 0,205 puncte. Le recapitulezi numai daca esti intebat. Reflexia textului este efectul nou, in valoare de 0,15 puncte; pentru HR mai poate exista cel mult o recuperare de 0,02 puncte daca profesoara accepta remedierea.

Componente in prim-plan

Descopera configuratiile si componentele recomandate de PC Forge pentru un sistem echilibrat. Treci cu cursorul peste imagine pentru a o vedea clar.



Duotone - efect deja prezentat: pseudo-elemente, mix-blend-mode si trecere la grayscale.

PC FORGE

Posiune pentru componente PC, periferice si configuratii pe masura.

Reflexia textului - efect nou: copie in ::after, rasturnare, blur si alungire la hover.

5.NOU Reflexia textului (0,15 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#efect-reflexie> | VS Code: views/pagini/index.ejs; resurse/css/stil.css

Ce cere exact: Titlul trebuie sa aiba o reflexie vizibila in browserele moderne, reflexia sa fie usor blurata, iar la hover sa se alungeasca.

Demonstratia, in ordinea corecta

Deschide sectiunea PC Forge cu reflexie.

Indica imaginea rasturnata a textului sub titlu.

Treci cu mouse-ul peste titlu si arata alungirea.

In CSS arata ::after, attr(data-text), scaleY, blur si mask-image.

Cum functioneaza in proiect

Titlul real pastreaza textul in HTML si valoarea data-text.

Pseudo-elementul ::after copiaza continutul prin attr(data-text), deci nu este nevoie de un al doilea heading.

scaleY(-1) rastoarna copia vertical, top: 100% o aseaza imediat sub text, iar transform-origin: top pastreaza contactul cu baza titlului.

filter: blur estompeaza reflexia, iar mask-image reduce treptat vizibilitatea spre partea de jos.

La hover, scaleY(-1.8) mareste reflexia pe verticala; tranzitia interpoleaza schimbarea.

Algoritmul pe care trebuie sa il intelegi

Copiaza textul in ::after.

Pozitioneaza copia sub titlu.

O rastoarna cu `scaleY(-1)`.

Aplica transparenta, blur si masca gradient.

La hover schimba scara la -1.8 si lasa transition sa anime diferenta.

Bucata-cheie: pseudo-elementul care formeaza reflexia:

```

1116 | /* Etapa 5 - efect-css-reflexie-text:
1117 | titlul propriu-zis caruia i se ataseaza reflexia prin ::after */
1118 | .titlu-reflexie {
1119 | position: relative;
1120 | display: inline-block;
1121 | margin: 2rem auto 4rem auto;
1122 | font-size: clamp(2.2rem, 6vw, 5rem);
1123 | line-height: 1;
1124 | color: var(--culoare-text-deschis);
1125 | text-transform: uppercase;
1126 | letter-spacing: 0.08em;
1127 | }
1128 |
1129 | /* Etapa 5 - efect-css-reflexie-text:
1130 | reflexia copiaza textul din data-text, il intoarce vertical cu scaleY(-1)
1131 | si il estompeaza cu blur si mask-image; sta sub titlu prin top: 100% */
1132 | .titlu-reflexie::after {
1133 | content: attr(data-text);
1134 | position: absolute;
1135 | left: 0;
1136 | top: 100%;
1137 | width: 100%;
1138 | color: var(--culoare-text-deschis);
1139 | transform: scaleY(-1);
1140 | transform-origin: top;
1141 | opacity: 0.45;
1142 | filter: blur(2px);
1143 | mask-image: linear-gradient(to bottom, rgba(0, 0, 0, 0.7), transparent);
1144 | -webkit-mask-image: linear-gradient(
1145 | to bottom,
1146 | rgba(0, 0, 0, 0.7),
1147 | transparent
1148 | );
1149 | transition:
1150 | transform 0.6s ease,
1151 | opacity 0.6s ease,
1152 | filter 0.6s ease;
1153 | }
1154 |
1155 | /* Etapa 5 - efect-css-reflexie-text:
1156 | la hover reflexia se alungeste prin scaleY(-1.8) */
1157 | .titlu-reflexie:hover::after {
1158 | transform: scaleY(-1.8);
1159 | opacity: 0.7;
1160 | filter: blur(3px);

```

Intrebari probabile si raspunsuri

De ce folosesti ::after?

Raspuns: Reflexia este decorativa si nu trebuie repetata ca text semantic pentru cititoarele de ecran.

De ce exista si data-text?

Raspuns: content: attr(data-text) poate copia in pseudo-element exact textul asociat titlului.

Care este diferenta dintre scaleY(-1) si rotate?

Raspuns: Valoarea negativa inverseaza vertical continutul ca intr-o oglinda; rotirea ar intoarce si orientarea laterala.

5.5 Backup timestamp, nume cu puncte si validarea galeriei (0,125 p)

Ce trebuie sa deschizi: Browser: Terminalul si folderul backup/resurse/css | VS Code: index.js; resurse/json/galerie.json; backup/resurse/css

Ce cere exact: Cele trei bonusuri cer copii CSS cu timp in nume, tratarea corecta a numelor cu mai multe puncte si verificarea datelor galeriei la pornire.

Demonstratia, in ordinea corecta

Arata un nume de backup care contine data si ora.

Indica path.parse si expresia care schimba numai extensia finala .scss.

Reporneste serverul si arata mesajul ca galeria a fost validata.

Explica verbal ca o imagine lipsa produce o eroare clara inainte de pornirea normala.

Cum functioneaza in proiect

Timestampul permite pastrarea mai multor versiuni fara suprascriere.

path.parse intelege separat folderul, numele si extensia; astfel tema.dark.scss devine tema.dark.css, nu tema.css.

valideazaGalerie verifica existenta folderului, structura listei si fiecare fisier mentionat.

Validarea timpurie transforma o imagine sparta greu de urmarit intr-un mesaj de pornire precis.

Algoritmul pe care trebuie sa il intelegi

Citeste si parseaza JSON-ul.

Verifica tipul proprietatilor si calea galeriei.

Construieste calea absoluta pentru fiecare imagine.

Acumuleaza sau arunca erori explicite.

Continua initializarea numai daca toate resursele sunt valide.

Bucata-cheie: validarea datelor si a fisierelor:

```

432 | function valideazaGalerie() {
433 |   const erori = [];
434 |   const caleGalerie = path.join(__dirname, String(obGlobal.obGalerie.cale_galerie ||
    "").replace(/^\//, ""));
435 |   if (!fs.existsSync(caleGalerie) || !fs.statSync(caleGalerie).isDirectory()) {
436 |     erori.push(`Folderul cale_galerie nu exista: ${caleGalerie}`);
437 |   } else {
438 |     for (const imagine of obGlobal.obGalerie.imagini || []) {
439 |       const caleImagine = path.join(caleGalerie, imagine.cale_imagine || "");
440 |       if (!imagine.cale_imagine || !fs.existsSync(caleImagine)) {
441 |         erori.push(`Imagine declarata in galerie.json, dar absenta de pe disc:
    ${caleImagine}`);
442 |       }
443 |     }
444 |   }
445 |   if (erori.length) erori.forEach((mesaj) => console.error(`[Etapa 5 - Bonus 5]
    ${mesaj}`));
446 |   else console.log("[Etapa 5 - Bonus 5] Datele galeriei au fost validate cu succes.");
447 |   return erori.length === 0;
448 | }
449 |
450 | // Miniaturile au aceeasi dimensiune si sunt decupate automat cu Sharp. Un fisier
451 | // este refacut numai daca lipseste sau daca imaginea originala este mai noua.
452 | /**
453 |  * Genereaza miniaturile WebP lipsa sau inechite cu Sharp.
454 |  * @returns {Promise<void>} Promise rezolvat dupa procesarea imaginilor.
455 |  */
456 | async function pregatesteMiniaturiGalerie() {
457 |   const folderOriginale = path.join(
458 |     __dirname,
459 |     "resurse",
460 |     "imagini",
461 |     "galerie",
462 |     "originale",
463 |   );
464 |   const folderMiniaturi = path.join(
465 |     __dirname,
466 |     "resurse",
467 |     "imagini",
468 |     "galerie",
469 |     "thumbnails",
470 |   );
471 |
472 |   creeazaFolderDacaNuExista(folderMiniaturi);
473 |
474 |   for (const imagine of obGlobal.obGalerie.imagini) {
475 |     const caleOriginal = path.join(folderOriginale, imagine.cale_imagine);
476 |     const numeMiniatura = path.parse(imagine.cale_imagine).name + ".webp";
477 |     const caleMiniatura = path.join(folderMiniaturi, numeMiniatura);
478 |
479 |     if (!fs.existsSync(caleOriginal)) {
480 |       console.warn("Imagine lipsa din galerie: " + caleOriginal);
481 |       continue;
482 |     }

```

```

483 |
484 | const trebuieGenerata =
485 | !fs.existsSync(caleMiniatura) ||
486 | fs.statSync(caleOriginal).mtimeMs > fs.statSync(caleMiniatura).mtimeMs;

```

Intrebari probabile si raspunsuri

De ce verifici la pornire?

Raspuns: Problema este descoperita imediat si serverul nu livreaza o pagina partial defecta.

De ce nu folosesti split('.')?

Raspuns: Un nume poate contine mai multe puncte; path.parse lucreaza corect cu ultima extensie.

Timestampul este doar decorativ?

Raspuns: Nu. El face numele unic si permite pastrarea istoricului.

ETAPA 6 - cerinte principale: 1,75 puncte

Ideea etapei: Produsele vin din PostgreSQL, sunt randate semantic cu EJS si apoi filtrate, sortate si calculate in browser. Bootstrap organizeaza controalele, iar tema se pastreaza.

Selecția PC Forge

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conexiuni	PCIe, USB-C, Ethernet
Adăugat	2 Mai 2025 (Sâmbătă)
Disponibil	Da



Platformă ATX pentru configurații AMD moderne, cu extensii rapide și traseu aerisit.
CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

★
🔊
🔥
📄

899.90 lei

Detalii →

Placa de baza Forge Mini-ITX

Scor	76/100
Culoare	negru
Conexiuni	PCIe, Wi-Fi, USB-C
Adăugat	18 Aprilie 2025 (Sâmbătă)
Disponibil	Da



Placa compacta pentru sisteme de dimensiuni reduse, fara compromisuri de conectivitate.
CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

★
🔊
🔥
📄

749.50 lei

Detalii →

Placa de baza Retro AT

Scor	25/100
Culoare	gri
Conexiuni	AT, PS/2
Adăugat	10 Noiembrie 2025 (Luni)
Disponibil	Nu



Model demonstrativ pentru prezentarea evolutiei arhitecturii calculatoarelor desktop.
CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

★
🔊
🔥
📄

299.00 lei

Detalii →

1 2 3 4

Program

Pagina /produse - cardurile respecta ordinea ceruta, au tabelul langa imagine si date provenite din PostgreSQL.

6.1 Datele si paginile produselor (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/produse` si `/produs/1` | VS Code: `database/*.sql`; `module/acces-produse.js`; `index.js`; `views/pagini/produse.ejs`; `views/pagini/produs.ejs`

Ce cere exact: Produsele trebuie pastrate in PostgreSQL, listate pe pagina generala si afisate individual pe o ruta care primeste id-ul produsului. In fiecare article ordinea ceruta este nume, tabel cu cinci tipuri de caracteristici, descriere si categorie; imaginea uniforma sta in dreapta tabelului, iar data este afisata romaneste intr-un element time.

Demonstratia, in ordinea corecta

Deschide `/produse` si indica ordinea din primul card: nume, tabel plus imagine, descriere si categorie.

In tabel identifica o valoare numerica, un sir, un sir cu valori multiple, data si booleanul.

Citeste data in forma 2 Mai 2026 (Sambata) si arata atributul datetime in cod.

Apasa Detalii pentru un produs si observa URL-ul `/produs/id`.

Schimba manual id-ul cu unul inexistent si arata tratarea erorii.

In cod urmareste ruta, functia de acces la date, helperul `formateazaDataProdus` si `res.render`.

Cum functioneaza in proiect

modulul `acces-produse` separa SQL-ul de rutele HTTP. Query-urile folosesc `$1`, `$2` pentru valori, reducand riscul de SQL injection.

Ruta `/produse` cere vectorul si il transmite in `res.render`. EJS repeta cate un article pentru fiecare produs.

In interiorul articolului, un grid local are tabelul in coloana stanga si imaginea patrata in coloana dreapta. Sub grid urmeaza descrierea si categoria, exact in ordinea ceruta.

Tabelul are scor numeric, culoare text, conectivitate ca lista de valori, data si disponibilitate booleana.

`formateazaDataProdus` desparte data ISO in an, luna si zi, foloseste vectorii cu luni si zile romanesti si produce textul vizibil. Atributul `datetime` pastreaza formatul standard pentru semantica HTML.

Ruta `/produs/:id` transforma parametrul in numar intreg pozitiv, cauta un singur produs si randeaza `produs.ejs` sau 404.

`data-*` de pe carduri reprezinta puntea dintre datele introduse de EJS si algoritmul JavaScript din browser.

Algoritmul pe care trebuie sa il intelegi

Browserul cere ruta.

Express apeleaza modulul de date.

PostgreSQL intoarce rows.

Serverul completeaza proprietatile calculate si pregateste helperul de data.

EJS produce article-ul in ordinea ceruta.

CSS Grid aseaza imaginea in dreapta tabelului.

Ruta dinamica afiseaza separat produsul selectat.

Bucata-cheie: structura ceruta pentru fiecare articol de produs:

```

112 | <% produse.forEach(function(produs, index) { %>
113 | <%# Etapa 7 - cerinta 16: acestea sunt carduri Bootstrap reale, animate din JS. %>
114 | <article class="card produs-card <%= produs.categorie %> <%= produs.subcategorie %>"
id="elem_<%= produs.id %>" data-produs-id="<%= produs.id %>" data-index-initial="<%= index
%>" data-nume="<%= produs.nume %>" data-descriere="<%= produs.descriere %>"
data-imagine="<%= produs.imagine %>" data-categorie="<%= produs.categorie %>"
data-subcategorie="<%= produs.subcategorie %>" data-pret="<%= produs.pret %>"
data-scor="<%= produs.scor_performanta %>" data-culoare="<%= produs.culoare %>"
data-conectivitate='<%= JSON.stringify(produs.conectivitate) %>' data-stoc="<%=
produs.in_stoc %>">
115 | <div class="card-body">
116 | <%# Etapa 6 - cerinta individuala, punctul 5: numele este primul continut al
entitatii. %>
117 | <h3 class="card-title"><a href="/produs/<%= produs.id %>"><%= produs.nume %></a></h3>
118 |
119 | <%# Etapa 6 - cerinta individuala, punctul 5: tabelul are exact cele cinci tipuri
cerute, iar imaginea este in dreapta lui. %>
120 | <div class="produs-card__detalii">
121 | <table class="table table-sm produs-specificatii"><tbody><tr><th>Scor</th><td><%=
produs.scor_performanta %>/100</td></tr><tr><th>Culoare</th><td><%= produs.culoare
%></td></tr><tr><th>Conexiuni</th><td><%= produs.conectivitate.join(", ")
%></td></tr><tr><th>Adăugat</th><td><time datetime="<%= produs.data_adaugare %>"><%=
formateazaDataProdus(produs.data_adaugare)
%></time></td></tr><tr><th>Disponibil</th><td><%= produs.in_stoc ? "Da" : "Nu"
%></td></tr></tbody></table>
122 | <div class="produs-card__imagine">
123 |  - <%= produs.descriere %>"
loading="lazy" />
124 | <div class="produs-card__badge-uri"><% if (produs.esteNou) { %><span class="badge
text-bg-primary">NOU</span><% } %><% if (produs.esteCelMaiIeftin) { %><span class="badge
text-bg-success">Cel mai ieftin din categorie</span><% } %></div>
125 | </div>
126 | </div>
127 |
128 | <%# Etapa 6 - cerinta individuala, punctul 5: descrierea si categoria urmeaza sub
tabel. %>
129 | <p class="card-text"><%= produs.descriere %></p>
130 | <p class=
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce folosesti time?

Raspuns: Elementul ofera o data standard masinilor prin datetime, dar utilizatorului ii arata forma romaneasca usor de citit.

De ce formatezi data pe server?

Raspuns: Toate cardurile primesc acelasi format inainte de trimiterea paginii si EJS ramane simplu.

De ce imaginea este intr-un grid cu tabelul?

Raspuns: Cerinta o solicita in dreapta tabelului; gridul pastreaza doua coloane clare si dimensiuni uniforme.

Ce este o ruta dinamica?

Raspuns: O ruta precum /produs/:id, unde :id este o valoare extrasa din URL.

Ce face await?

Raspuns: Suspenda functia async pana cand Promise-ul query-ului se rezolva, fara a bloca intregul server.

De ce separi acces-produse.js?

Raspuns: Ruta se ocupa de HTTP, iar modulul de date de SQL; fiecare responsabilitate poate fi verificata separat.

6.2 Cele opt filtre si validarea (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: views/pagini/produse.ejs; resurse/js/produse.js

Ce cere exact: Filtrele folosesc text, range, datalist, radio, checkbox, textarea, select simplu si select multiplu. Valorile nepotrivite trebuie ascunse, nu sterse din DOM, iar inputurile afectate trebuie validate.

Demonstratia, in ordinea corecta

Testeaza pe rand numele, scorul, culoarea, stocul, categoria, descrierea, subcategoria si conectivitatea.

Combina doua filtre si explica faptul ca toate conditiile sunt legate prin AND.

Introdu doua caractere in textarea si arata starea is-invalid; la trei caractere eroarea dispare.

Apasa Resetare si confirma revenirea la ordinea si valorile initiale.

Cum functioneaza in proiect

obteneStareFiltre citeste toate controalele si creeaza un singur obiect coerent.

aplicaFiltrare parcurge cardurile deja existente si compara valorile inputurilor cu dataset-ul produsului.

includes este folosit pentru text, comparatia numerica pentru scor, iar every cere ca toate optiunile de conectivitate selectate sa existe la produs.

hidden ascunde cardul, dar il pastreaza in DOM pentru resetare si filtrari ulterioare.

valideazaInputuri foloseste expresii regulate, setCustomValidity, is-invalid si reportValidity.

Algoritmul pe care trebuie sa il intelegi

Citeste si normalizeaza valorile filtrelor.

Verifica validitatea campurilor.

Pentru fiecare produs calculeaza opt conditii booleene.

Combina conditiile cu operatorul &&.

Salveaza eligibilitatea, apoi actualizeaza paginarea, mesajul si contorul.

Bucata-cheie: cele opt conditii ale filtrarii:

```

121 | function aplicaFiltrare(afiseazaMesaje = false) {
122 |   valoareScor.value = inpScor.value;
123 |   if (!valideazaInputuri(afiseazaMesaje)) return false;
124 |
125 |   const filtru = {
126 |     nume: normalizeazaText(inpNume.value.trim()),
127 |     scor: Number(inpScor.value),
128 |     culoare: inpCuloare.value.trim().toLowerCase(),
129 |     stoc: document.querySelector('input[name="stoc"]:checked').value,
130 |     categorii: valoriSelectate(".filtru-categorie"),
131 |     descriere: normalizeazaText(inpDescriere.value.trim()),
132 |     subcategorie: selectSubcategorie.value,
133 |     conectivitate: optiuniMultiple(selectConectivitate),
134 |   };
135 |
136 |   let numarVizibile = 0;
137 |   produse.forEach(function (produs) {
138 |     const conexiuniProdus = JSON.parse(produs.dataset.conectivitate);
139 |     const corespunde =
140 |       normalizeazaText(produs.dataset.nume).includes(filtru.nume) &&
141 |       Number(produs.dataset.scor) >= filtru.scor &&
142 |       (!filtru.culoare || produs.dataset.culoare === filtru.culoare) &&
143 |       (filtru.stoc === "toate" || (filtru.stoc === "da" && produs.dataset.stoc === "true")
144 |       || (filtru.stoc === "nu" && produs.dataset.stoc === "false")) &&
145 |       (filtru.categorii.length === 0 ||
146 |       filtru.categorii.includes(produs.dataset.categorie)) &&
147 |       normalizeazaText(produs.dataset.descriere).includes(filtru.descriere) &&
148 |       (!filtru.subcategorie || produs.dataset.subcategorie === filtru.subcategorie) &&
149 |       (filtru.conectivitate.length === 0 || filtru.conectivitate.every((valoare) =>
150 |       conexiuniProdus.includes(valoare)));
151 |     // Etapa 6 - Bonus 6: un produs fixat suprascrie filtrul, iar cel ascuns
152 |     // in sessionStorage ramane invizibil numai in tabul curent.
153 |     produs.dataset.ascunsTemporar = "false";
154 |     const eligibil = (corespunde || produs.classList.contains("produs-card--fixat")) &&
155 |     !ascunseTab.has(produs.dataset.produsId);
156 |     produs.dataset.corespunde = String(eligibil);
157 |     if (eligibil) numarVizibile++;
158 |   });
159 |   // Etapa 6 - Bonus 3 si Bonus 15.
160 |   mesajFaraProduse.hidden = numarVizibile !== 0;
161 |   numarProduse.textContent = `${numarVizibile} ${numarVizibile === 1 ? "produs" :
162 |   "produse"}`;
163 |   paginaCurenta = 1;
164 |   actualizeazaPaginare();
165 |   salveazaStareFi
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce folosesti dataset?

Raspuns: EJS pune datele produsului in atribute data-*, iar JavaScript le poate citi fara o noua cerere la server.

De ce hidden si nu remove?

Raspuns: remove ar elimina nodul; hidden permite readucerea lui la resetare.

Validarea din browser este suficienta pentru baza de date?

Raspuns: Nu. Este pentru experienta utilizatorului; datele trimise serverului trebuie validate din nou.

6.3 Sortarea, calculul si resetarea (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Sortarea foloseste doua chei, calculul opereaza numai pe produsele vizibile, iar resetarea cere confirmare si reface ordinea initiala.

Demonstratia, in ordinea corecta

Alege pret si nume, apoi sorteaza crescator si descrescator.

Alege suma, media, minim sau maxim si arata mesajul care dispare dupa doua secunde.

Aplica filtre, apoi apasa Resetare si raspunde OK.

Arata revenirea cheilor, filtrelor si cardurilor la starea initiala.

Cum functioneaza in proiect

Comparatorul intoarce diferenta primei chei. Numai la egalitate compara a doua cheie.

sens este 1 pentru crescator si -1 pentru descrescator, evitand duplicarea algoritmului.

append muta nodurile existente in noua ordine; nu le cloneaza.

Calculul filtreaza cardurile neascunse, transforma preturile in Number si foloseste reduce, Math.min sau Math.max.

indexInitial pastreaza pozitia initiala necesara resetarii.

Algoritmul pe care trebuie sa il intelegi

Valideaza inputurile.

Construieste un vector nou din produse.

Sorteaza dupa cheia principala, apoi secundara.

Muta cardurile in DOM.

Pentru calcul extrage preturile vizibile si creeaza mesajul cu createElement.

La resetare sterge starile persistente si sorteaza dupa indexInitial.

Bucata-cheie: comparatorul, calculul si inceputul resetarii:

```

190 | function valoareSortare(produs, cheie) {
191 | if (cheie === "raport") return Number(produs.dataset.scor) /
Number(produs.dataset.pret);
192 | if (cheie === "pret" || cheie === "scor") return Number(produs.dataset[cheie]);
193 | return normalizeazaText(produs.dataset[cheie]);
194 | }
195 |
196 | function comparaValori(a, b) {
197 | if (typeof a === "number" && typeof b === "number") return a - b;
198 | return String(a).localeCompare(String(b), "ro", { numeric: true });
199 | }
200 |
201 | // Etapa 6 - punctul 8.2 si Bonus 8: sortare dupa doua chei alese.
202 | function sorteazaProduse(sens) {
203 | if (!valideazaInputuri(true)) return;
204 | const cheie1 = document.getElementById("cheie-sortare-1").value;
205 | const cheie2 = document.getElementById("cheie-sortare-2").value;
206 | const sortate = [...produse].sort(function (produsA, produsB) {
207 | const primaComparatie = comparaValori(valoareSortare(produsA, cheie1),
valoareSortare(produsB, cheie1));
208 | if (primaComparatie !== 0) return primaComparatie * sens;
209 | return comparaValori(valoareSortare(produsA, cheie2), valoareSortare(produsB,
cheie2)) * sens;
210 | });
211 | sortate.forEach((produs) => lista.append(produs));
212 | produse.forEach((produs) => { produs.dataset.ascunsTemporar = "false"; });
213 | actualizeazaPaginare();
214 | }
215 |
216 | // Etapa 6 - punctul 8.3: rezultatul este creat dinamic si sters dupa 2 secunde.
217 | function calculeazaPreturi() {
218 | if (!valideazaInputuri(true)) return;
219 | const preturi = produse.filter((produs) => !produs.hidden).map((produs) =>
Number(produs.dataset.pret));
220 | if (!preturi.length) {
221 | afiseazaCalcul("Nu există produse pentru calcul.");
222 | return;
223 | }
224 | const tip = document.getElementById("tip-calcul").value;
225 | const suma = preturi.reduce((total, pret) => total + pret, 0);
226 | const rezultate = { suma: suma, medie: suma / preturi.length, minim:
Math.min(...preturi), maxim: Math.max(...preturi) };
227 | const etichete = { suma: "Suma", medie: "Media", minim: "Minimul", maxim: "Maximul" };
228 | afiseazaCalcul(`${etichete[tip]} prețurilor: ${rezultate[tip].toFixed(2)} lei`);
229 | }
230 |
231 | function afiseazaCalcul(text) {
232 | document.querySelectorAll(".rezultat-calcul").forEach((element) => element.remove());
233 |
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce comparatorul poate intoarce un numar negativ?

Raspuns: Negativ inseamna ca primul element trebuie pus inainte, pozitiv dupa, iar zero inseamna egalitate.

Ce face reduce?

Raspuns: Combina toate valorile intr-un acumulator; aici aduna preturile.

De ce rezultatul dispare?

Raspuns: Cerinta cere un element creat dinamic, vizibil doua secunde si apoi sters cu setTimeout.

6.4 Stilizarea Bootstrap a inputurilor (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: `views/pagini/produse.ejs`; `resurse/sass/custom.scss`; `resurse/css/custom.css`

Ce cere exact: Butoanele si inputurile folosesc Bootstrap, textarea are floating label si stare invalida, radio/checkbox au aspect de toggle, controalele sunt asezate in grid, tema foloseste un control Bootstrap, iar range-ul este personalizat prin variabile Sass.

Demonstratia, in ordinea corecta

Indica butoanele cu iconuri si micsoreaza fereastra pentru a ramane doar iconurile.

Arata floating label-ul textarea si provoaca starea invalida.

Apasa radio-urile de stoc si observa trecerea outline-plin.

Arata coloanele Bootstrap si bulina marita a range-ului.

Cum functioneaza in proiect

form-control, form-select, form-range si btn sunt clase Bootstrap, nu reguli imitate manual.

btn-check ascunde vizual inputul, dar labelul conectat prin for devine buton toggle accesibil.

row si col-* definesc coloanele la breakpointuri.

Variabilele Sass pentru range schimba dimensiunea, culoarea thumbului si pista inaintea compilarii Bootstrap.

Algoritmul pe care trebuie sa il intelegi

EJS produce controale cu clasele frameworkului.

Bootstrap aplica aspectul de baza si gridul.

JavaScript adauga is-invalid cand valoarea nu respecta regula.

CSS-ul customizat furnizeaza paleta si dimensiunile PC Forge.

Bucata-cheie: cele opt controale si butoanele Bootstrap:

```

23 | <div class="row g-3" id="zona-filtre">
24 | <div class="col-12 col-md-6 col-xl-4">
25 | <%# Etapa 6 - Bonus 1, input text: maxlength si exemplul provin din produsele bazei de
    | date. %>
26 | <label class="form-label" for="inp-nume">Numele produsului (maximum <%=
    | filtre.lungimeMaximaNume %> caractere)</label>
27 | <input class="form-control" type="text" id="inp-nume" maxlength="<%=
    | filtre.lungimeMaximaNume %>" placeholder="Exemplu: <%= filtre.exempluNume %>"
    | autocomplete="off" />
28 | <div class="invalid-feedback">Folosește numai litere, spații și cratimă.</div>
29 | </div>
30 |
31 | <div class="col-12 col-md-6 col-xl-4">
32 | <label class="form-label d-flex justify-content-between" for="inp-scor"><span>Scor
    | minim de performanță</span><output id="valoare-scor" for="inp-scor"><%= filtre.scorMinim
    | %></output></label>
33 | <!-- Etapa 6 - cerinta 6b: min/max si valoarea selectata sunt generate din date. -->
34 | <input class="form-range" type="range" id="inp-scor" min="<%= filtre.scorMinim %>"
    | max="<%= filtre.scorMaxim %>" value="<%= filtre.scorMinim %>" />
35 | <div class="range-limite"><span><%= filtre.scorMinim %></span><span><%=
    | filtre.scorMaxim %></span></div>
36 | </div>
37 |
38 | <div class="col-12 col-md-6 col-xl-4">
39 | <label class="form-label" for="inp-culoare">Culoare</label>
40 | <input class="form-control" type="text" id="inp-culoare" list="lista-culori"
    | placeholder="Orice culoare" autocomplete="off" />
41 | <datalist id="lista-culori"><% filtre.culori.forEach(function(culoare) { %><option
    | value="<%= culoare %>"></option><% }></datalist>
42 | <div class="invalid-feedback">Alege o culoare existentă din listă.</div>
43 | </div>
44 |
45 | <fieldset class="col-12 col-md-6 col-xl-4">
46 | <legend class="form-label">Disponibilitate</legend>
47 | <!-- Etapa 6 - task Bootstrap 0.3c: radio buttons ca toggle buttons. -->
48 | <%# Etapa 6 - Bonus 1, radio: optiunile sunt construite din valorile booleene existente
    | in baza. %>
49 | <div class="btn-group w-100" role="group" aria-label="Disponibilitate">
50 | <% filtre.optiuniStoc.forEach(function(optiune, index) { %>
51 | <input class="btn-check" ty
    | ... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

Cum dovedesti ca este Bootstrap?

Raspuns: Arat clasele form-control, form-range, btn-check, row, col si btn, plus variabilele compilate din Sass.

De ce labelul trebuie legat de input?

Raspuns: Clickul pe label activeaza controlul, iar tehnologiile asistive cunosc numele campului.

De ce textul butoanelor dispare pe mobil?

Raspuns: Iconul pastreaza actiunea recognoscibila si economiseste spatiu; textul ramane accesibil semantic.

6.5 Tema light/dark persistenta (0,20 p)

Ce trebuie sa deschizi: Browser: Orice pagina; selectorul de tema din header | VS Code: resurse/js/tema.js; views/fragmente/header.ejs; resurse/css/teme.css

Ce cere exact: Utilizatorul schimba tema, iar alegerea se pastreaza in localStorage si se aplica pe paginile urmatoare.

Demonstratia, in ordinea corecta

Alege tema intunecata.

Navigheaza pe alta pagina si apoi reincarca.

Arata ca tema ramane activa.

In DevTools sau cod indica atributul temei si cheia din localStorage.

Cum functioneaza in proiect

localStorage pastreaza perechi cheie-valoare chiar dupa inchiderea paginii.

Scriptul citeste preferinta devreme si seteaza atributul/clasa documentului.

CSS foloseste acel selector pentru a schimba variabilele de culoare fara a dubla structura HTML.

Algoritmul pe care trebuie sa il intelegi

Citeste tema salvata sau foloseste valoarea implicita.

Aplica tema pe elementul document.

La schimbarea controlului reaplica tema.

Salveaza numele temei pentru vizitele urmatoare.

Bucata-cheie: citirea, aplicarea si persistenta temei:

```

1 | // Etapa 6 - task tema light/dark:
2 | // tema se memoreaza in localStorage si se aplica pe toate paginile site-ului.
3 | (function () {
4 |   const cheieTema = "pc-forge-tema";
5 |   const temaServer = window.pcForgeUtilizator?.tema;
6 |   const temaSalvata = temaServer || localStorage.getItem(cheieTema);
7 |   // Etapa 6 - Bonus 2: sunt acceptate trei teme persistente.
8 |   const temaInitiala = ["light", "dark", "contrast"].includes(temaSalvata) ? temaSalvata :
   "light";
9 |
10 |   function aplicaTema(tema) {
11 |     document.documentElement.dataset.tema = tema;
12 |     const comutator = document.getElementById("comutator-tema");
13 |     const selector = document.getElementById("selector-tema");
14 |     if (selector) selector.value = tema;
15 |     if (comutator) {
16 |       comutator.checked = tema === "dark";
17 |       comutator.setAttribute("aria-label", tema === "dark" ? "Activează tema luminoasă" :
   "Activează tema întunecată");
18 |     }
19 |   }
20 |
21 |   aplicaTema(temaInitiala);
22 |   const comutator = document.getElementById("comutator-tema");
23 |   const selector = document.getElementById("selector-tema");
24 |   function memoreazaTema(tema) {
25 |     localStorage.setItem(cheieTema, tema);
26 |     aplicaTema(tema);
27 |     if (window.pcForgeUtilizator) fetch("/api/tema", { method: "POST", headers: {
   "Content-Type": "application/json" }, body: JSON.stringify({ tema: tema }) });
28 |   }
29 |   if (comutator) {
30 |     comutator.addEventListener("change", function () {
31 |       const tema = comutator.checked ? "dark" : "light";
32 |       memoreazaTema(tema);
33 |     });
34 |   }
35 |   if (selector) selector.addEventListener("change", () =>
   memoreazaTema(selector.value));
36 | })();

```

Intrebari probabile si raspunsuri

localStorage ajunge automat la server?

Raspuns: Nu. Ramane in browser. Pentru preferinta contului exista separat salvarea prin server.

De ce folosesti variabile CSS?

Raspuns: Aceeasi componenta isi schimba culorile prin valori comune, fara reguli duplicate pentru fiecare element.

ETAPA 6 - bonusurile selectate: 6 puncte

Ideea etapei: Aceste bonusuri extind acelasi sistem de produse. La pachetul vizual de 4,05 puncte se adauga Bonusul 1, Bonusul 13 si Bonusul 17, in valoare de 1,95 puncte.

Selecția PC Forge

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conexiuni	PCIe, USB-C, Ethernet
Adăugat	2 Mai 2025 (Sâmbătă)
Disponibil	Da

Platformă ATX pentru configurații AMD moderne, cu extensii rapide și traseu aerisit.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

899.90 lei

Detalii →

Placa de baza Forge Mini-ITX

Scor	76/100
Culoare	negru
Conexiuni	PCIe, Wi-Fi, USB-C
Adăugat	18 Aprilie 2025 (Sâmbătă)
Disponibil	Da

Placa compacta pentru sisteme de dimensiuni reduse, fara compromisuri de conectivitate.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

749.50 lei

Detalii →

Placa de baza Retro AT

Scor	25/100
Culoare	gri
Conexiuni	AT, PS/2
Adăugat	10 Noiembrie 2025 (Luni)
Disponibil	Nu

Model demonstrativ pentru prezentarea evolutiei arhitecturii calculatoarelor desktop.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

299.00 lei

Detalii →

1 2 3 4

Program

Cardurile contin date, marcaje si actiuni folosite de mai multe bonusuri.

CONFIGURATOR PC FORGE

20 produse

Filtrează configurația

Numele produsului (maximum 32 caractere)

Exemplu: Placă de bază Forge B650

Scor minim de performanță

25 25 94

Culoare

Orice culoare

Disponibilitate

Taate In stoc Indisponibil

Categorii

componente monitoare periferice racire stocare

Descriere (maximum 87 caractere)

Subcategorie

Orice subcategorie

Conectivitate (selecție multiplă)

4-pin ARGB AT

Salvează filtrele

Filtrează

Filtre server

Raport scor/preț

Subcategorie

17 Crescător

47 Descrescător

Suma prețurilor

Program

Calculează

Resetează

Bonusul 1 - limitele si optiunile filtrelor sunt construite din datele produselor, nu din valori inventate in HTML.

6.B1 Atributele si optiunile inputurilor generate din date (maximum 0,80 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/produse` | VS Code: `index.js`; `views/pagini/produse.ejs`; `module/acces-produse.js`

Ce cere exact: Pentru fiecare dintre cele opt tipuri de input, atributele sau optiunile relevante sunt calculate din produsele citite din PostgreSQL. Se acorda cate 0,10 puncte pentru fiecare tip demonstrat corect.

Demonstratia, in ordinea corecta

Deschide `/produse` si indica cele opt controale: text, range, datalist, radio, checkbox, textarea, select simplu si select multiplu.

Pentru text arata `maxlength` si exemplul de produs generate din date.

La range arata min/max calculate din scoruri; la datalist arata culorile.

La radio arata optiunile de stoc obtinute din valorile booleene, iar la checkbox categoriile distincte.

La textarea arata `maxlength` calculat din descrieri; apoi deschide subcategoriile si conectivitatile celor doua selecturi.

In `index.js` indica formarea obiectului filtre, apoi in `produse.ejs` arata interpolările EJS si buclele `forEach`.

Cum functioneaza in proiect

Ruta `/produse` cere vectorul de produse din PostgreSQL.

Inputul text primeste lungimea maxima a numelor si un exemplu din primul produs.

`map` extrage scorurile, iar `Math.min` si `Math.max` determina limitele range-ului.

Culorile alimenteaza datalistul; valorile booleene distincte devin optiunile radio; categoriile devin checkboxuri.

Textarea primeste lungimea maxima a descrierilor.

Set elimina duplicatele pentru subcategorii si conectivitati, apoi selecturile sunt generate prin bucle EJS.

Serverul trimite un singur obiect filtre catre `res.render`, iar browserul primeste HTML adaptat datelor curente.

Algoritmul pe care trebuie sa il intelegi

Citeste produsele din baza de date.

Calculeaza lungimile maxime pentru text si textarea.

Calculeaza limitele numerice pentru range.

Extrage listele distincte pentru datalist, radio, checkbox si selecturi.

Elimina duplicatele si sorteaza listele.

Transmite obiectul filtre template-ului.

Genereaza atributele si optiunile prin EJS.

Bucata-cheie: construirea completa a obiectului filtre:

```

861 | const scoruri = produse.map((produs) => produs.scor_performanta);
862 | const subcategorii = [...new Set(produse.map((produs) =>
produs.subcategorie))].sort();
863 | const culori = [...new Set(produse.map((produs) => produs.culoare))].sort();
864 | const conexiuni = [
865 |   ...new Set(produse.flatMap((produs) => produs.conectivitate)),
866 | ].sort();
867 | const lungimeMaximaNume = Math.max(
868 |   1,
869 |   ...produse.map((produs) => produs.num.length),
870 | );
871 | const lungimeMaximaDescriere = Math.max(
872 |   3,
873 |   ...produse.map((produs) => produs.descriere.length),
874 | );
875 | const optiuniStoc = [
876 |   { valoare: "toate", eticheta: "Toate" },
877 |   ...[...new Set(produse.map((produs) => Boolean(produs.in_stoc)))]
878 |     .sort((a, b) => Number(b) - Number(a))
879 |     .map((inStoc) => ({
880 |       valoare: inStoc ? "da" : "nu",
881 |       eticheta: inStoc ? "In stoc" : "Indisponibil",
882 |     })),
883 | ];
884 |
885 | res.render("pagini/produse", {
886 |   ip: req.ip,
887 |   produse: produse,
888 |   categorieCurenta: categorie,
889 |   etichetaCategorie: categorie
890 |     ? eticheteCategorii[categorie]
891 |     : "Toate produsele",
892 |   filtre: {
893 |     scorMinim: scoruri.length ? Math.min(...scoruri) : 1,
894 |     scorMaxim: scoruri.length ? Math.max(...scoruri) : 100,
895 |     subcategorii: subcategorii,
896 |     culori: culori,
897 |     conexiuni: conexiuni,
898 |     lungimeMaximaNume: lungimeMaximaNume,
899 |     lungimeMaximaDescriere: lungimeMaximaDescriere,
900 |     exempluNume: produse[0]?.nume || "Produs PC Forge",
901 |     optiuniStoc: optiuniStoc,
902 |     categorii: [
903 |       ...new Set(produse.map((produs) => produs.categorie)),
904 |     ].sort(),

```

Intrebari probabile si raspunsuri

Ce inseamna generat din baza de date?

Raspuns: Nu scriu manual valori precum scor minim 1 sau lista culorilor. Ele sunt deduse prin program din produsele returnate de PostgreSQL.

De ce folosesti Set?

Raspuns: Mai multe produse pot avea aceeasi culoare sau subcategorie; Set pastreaza fiecare valoare o singura data.

Ce rol are operatorul spread?

Raspuns: Transforma colectia Set inapoi intr-un vector pe care il pot sorta si transmite template-ului.

EJS interogheaza direct PostgreSQL?

Raspuns: Nu. Modulul de date interogheaza baza, ruta pregateste valorile, iar EJS doar genereaza HTML din obiectul primit.

Cum demonstrezi cele 0,80 puncte?

Raspuns: Numesc fiecare dintre cele opt tipuri de control si indic pentru fiecare atributul sau lista alimentata din obiectul filtre.

Atentie la prezentare: Punctajul este fractionat. Nu spune doar ca intreaga zona este dinamica; arata separat ce informatie generata primeste fiecare tip de input.

6.B2 Trei teme persistente (0,15 p)

Ce trebuie sa deschizi: Browser: Selectorul din header, pe / si /produse | VS Code: resurse/js/tema.js; resurse/css/teme.css; views/fragmente/header.ejs

Ce cere exact: Alegerea nu se limiteaza la light/dark: proiectul ofera Luminoasa, Intunecata si Contrast Forge, toate persistente.

Demonstratia, in ordinea corecta

Comuta succesiv cele trei teme.

Pentru fiecare, indica schimbarea contrastului si a suprafetelor.

Lasa Contrast Forge activa, reincarca si navigheaza.

Cum functioneaza in proiect

Bonusul extinde mecanismul obligatoriu; cheia localStorage stocheaza numele uneia dintre valorile permise.

Tema Contrast Forge nu este doar inversarea culorilor, ci o a treia configuratie definita explicit.

Algoritmul pe care trebuie sa il intelegi

Valideaza alegerea in lista temelor permise.

Actualizeaza documentul.

Persistenta pastreaza alegerea.

Bucata-cheie: aceeaasi functie gestioneaza toate cele trei teme:

```

1 | // Etapa 6 - task tema light/dark:
2 | // tema se memoreaza in localStorage si se aplica pe toate paginile site-ului.
3 | (function () {
4 |   const cheieTema = "pc-forge-tema";
5 |   const temaServer = window.pcForgeUtilizator?.tema;
6 |   const temaSalvata = temaServer || localStorage.getItem(cheieTema);
7 |   // Etapa 6 - Bonus 2: sunt acceptate trei teme persistente.
8 |   const temaInitiala = ["light", "dark", "contrast"].includes(temaSalvata) ? temaSalvata :
   "light";
9 |
10 |   function aplicaTema(tema) {
11 |     document.documentElement.dataset.tema = tema;
12 |     const comutator = document.getElementById("comutator-tema");
13 |     const selector = document.getElementById("selector-tema");
14 |     if (selector) selector.value = tema;
15 |     if (comutator) {
16 |       comutator.checked = tema === "dark";
17 |       comutator.setAttribute("aria-label", tema === "dark" ? "Activează tema luminoasă" :
   "Activează tema întunecată");
18 |     }
19 |   }
20 |
21 |   aplicaTema(temaInitiala);
22 |   const comutator = document.getElementById("comutator-tema");
23 |   const selector = document.getElementById("selector-tema");
24 |   function memoreazaTema(tema) {
25 |     localStorage.setItem(cheieTema, tema);
26 |     aplicaTema(tema);
27 |     if (window.pcForgeUtilizator) fetch("/api/tema", { method: "POST", headers: {
   "Content-Type": "application/json" }, body: JSON.stringify({ tema: tema }) });
28 |   }
29 |   if (comutator) {
30 |     comutator.addEventListener("change", function () {
31 |       const tema = comutator.checked ? "dark" : "light";
32 |       memoreazaTema(tema);
33 |     });
34 |   }
35 |   if (selector) selector.addEventListener("change", () =>
   memoreazaTema(selector.value));
36 | })();

```

Intrebari probabile si raspunsuri

Este acelasi punctaj ca light/dark?

Raspuns: Nu. Light/dark este cerinta de baza de 0,20; a treia tema aduce bonusul separat de 0,15.

6.B3+B15 Mesajul fara rezultate si contorul (0,10 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Dupa fiecare filtrare se afiseaza numarul produselor eligibile; la zero, lista este inlocuita vizual de un mesaj relevant.

Demonstratia, in ordinea corecta

Observa numarul initial.

Scrie un nume imposibil si arata zero plus mesajul.

Sterge textul si arata revenirea numarului.

Cum functioneaza in proiect

numarVizibile este incrementat in aceeași buclă ce evaluează filtrarea, deci nu se parcurge inutil lista din nou.

hidden este comutat pe mesaj în funcție de comparația cu zero, iar textContent evita interpretarea conținutului ca HTML.

Algoritmul pe care trebuie sa il intelegi

Porneste contorul de la zero.

Incrementează pentru fiecare produs eligibil.

Actualizează textul singular/plural.

Ascunde sau afișează mesajul.

Bucata-cheie: numărarea și actualizarea celor două elemente:

```

145 | normalizeazaText(produs.dataset.descriere).includes(filtru.descriere) &&
146 | (!filtru.subcategorie || produs.dataset.subcategorie === filtru.subcategorie) &&
147 | (filtru.conectivitate.length === 0 || filtru.conectivitate.every((valoare) =>
conexiuniProdus.includes(valoare)));
148 |
149 | // Etapa 6 - Bonus 6: un produs fixat suprascrie filtrul, iar cel ascuns
150 | // in sessionStorage ramane invizibil numai in tabul curent.
151 | produs.dataset.ascunsTemporar = "false";
152 | const eligibil = (corespunde || produs.classList.contains("produs-card--fixat")) &&
!ascunseTab.has(produs.dataset.produsId);
153 | produs.dataset.corespunde = String(eligibil);
154 | if (eligibil) numarVizibile++;
155 | });
156 |
157 | // Etapa 6 - Bonus 3 si Bonus 15.
158 | mesajFaraProduse.hidden = numarVizibile !== 0;
159 | numarProduse.textContent = `${numarVizibile} ${numarVizibile === 1 ? "produs" :
"produse"}`;
160 | paginaCurenta = 1;
161 | actualizeazaPaginare();
162 | salveazaStareFiltre();
163 | return true;
164 | }
165 |
166 | // Etapa 6 - Bonus 5: genereaza ceil(N/K) butoane si afiseaza intervalul paginii P.

```


Intrebari probabile si raspunsuri

De ce `textContent` si nu `innerHTML`?

Raspuns: Aici avem text simplu; `textContent` evita parsarea HTML si este alegerea mai sigura.

Produsul este sters?

Raspuns: Nu, doar ascuns. Poate reveni imediat dupa schimbarea filtrului.

6.B4 Filtrarea imediata (maximum 0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#)

Ce cere exact: Fiecare dintre cele opt filtre reaplica algoritmul imediat, la input pentru text/range/textarea si la change pentru controalele discrete.

Demonstratia, in ordinea corecta

Tasteaza lent in nume si observa actualizarea la fiecare caracter.

Muta range-ul si urmareste scorul si cardurile.

Bifeaza categoria si radio-ul fara a apasa Filtreaza.

Cum functioneaza in proiect

input se declanseaza in timpul tastarii sau deplasarii range-ului. change se potriveste selecturilor, radio-urilor si checkboxurilor.

matches alege evenimentul eficient pentru fiecare control. Callbackul comun apeleaza `aplicaFiltrare(false)`.

Bonusul depinde de functionarea corecta a celor opt filtre; nu este suficienta doar atasarea evenimentului.

Algoritmul pe care trebuie sa il intelegi

Selecteaza toate controalele din zona filtrelor.

Pentru fiecare determina tipul.

Ataseaza input sau change.

La eveniment ruleaza validarea discreta si filtrarea.

Bucata-cheie: atasarea evenimentului potrivit tuturor filtrelor:

```

356 | salveazaFiltre.addEventListener("change", function () {
357 |   if (salveazaFiltre.checked) salveazaStareFiltre();
358 |   else localStorage.removeItem(cheieFiltrePersistente);
359 | });
360 |
361 | // Etapa 6 - Bonus 4: filtrarea se actualizeaza imediat la schimbarea inputurilor.
362 | document.querySelectorAll("#zona-filtre input, #zona-filtre textarea, #zona-filtre
select").forEach(function (input) {
363 |   const eveniment = input.matches('input[type="text"], input[type="range"], textarea') ?
"input" : "change";
364 |   input.addEventListener(eveniment, () => aplicaFiltrare(false));
365 | });
366 | restaureazaStareFiltre();
367 | aplicaFiltrare();
368 | afiseazaComparatie();
369 | animeazaCarduriBootstrap();
370 | });

```

Intrebari probabile si raspunsuri

De ce nu folosesti click peste tot?

Raspuns: Click nu descrie toate modificarile, de exemplu tastarea sau selectia cu tastatura.

De ce false la aplicaFiltrare?

Raspuns: Filtrarea live nu trebuie sa afiseze agresiv mesaje native la fiecare tasta; butoanele explicite pot folosi true.

6.B5 Paginarea (0,50 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Pentru N produse si $K=6$ produse pe pagina se genereaza $\text{ceil}(N/K)$ butoane, iar pagina P afiseaza intervalul corect.

Demonstratia, in ordinea corecta

Reseteaza filtrele pentru a avea suficiente produse.

Apasa pagina 2 si indica schimbarea celor sase carduri.

Aplica un filtru si arata recalcularea numarului de pagini.

In cod indica `Math.ceil` si `slice`.

Cum functioneaza in proiect

`Math.ceil` rotunjeste in sus deoarece si un rest de produs necesita o pagina noua.

Indicii intervalului sunt $(P-1)*K$ inclusiv si $P*K$ exclusiv, exact conventia `slice`.

Butoanele sunt create dinamic cu `createElement`; pagina activa primeste clasa `Bootstrap active`.

Paginarea se aplica dupa filtrare, asupra produselor eligibile, nu asupra intregii liste.

Algoritmul pe care trebuie sa il intelegi

Construiești vectorul eligibile.

Calculează numărPagini cu Math.ceil.

Limitează pagina curentă la noul număr.

Ascunde toate cardurile și afișează slice-ul paginii.

Reconstruiești butoanele și ascultătorii lor.

Bucată-cheie: formula și intervalul paginii:

```
167 | function actualizeazaPaginare() {
168 |   const eligibile = produse.filter((produs) => produs.dataset.corespunde === "true" &&
produs.dataset.ascunsTemporar !== "true");
169 |   const numarPagini = Math.max(1, Math.ceil(eligibile.length / produsePePagina));
170 |   paginaCurenta = Math.min(paginaCurenta, numarPagini);
171 |   produse.forEach((produs) => { produs.hidden = true; });
172 |   eligibile.slice((paginaCurenta - 1) * produsePePagina, paginaCurenta *
produsePePagina).forEach((produs) => { produs.hidden = false; });
173 |   const container = document.getElementById("paginare-produse");
174 |   container.replaceChildren();
175 |   if (numarPagini <= 1) return;
176 |   for (let pagina = 1; pagina <= numarPagini; pagina++) {
177 |     const element = document.createElement("li");
178 |     element.className = `page-item${pagina === paginaCurenta ? " active" : ""}`;
179 |     const buton = document.createElement("button");
180 |     buton.className = "page-link";
181 |     buton.type = "button";
182 |     buton.textContent = pagina;
183 |     buton.setAttribute("aria-label", `Pagina ${pagina}`);
184 |     buton.addEventListener("click", () => { paginaCurenta = pagina;
actualizeazaPaginare(); document.getElementById("titlu-lista-produse").scrollIntoView({
behavior: "smooth" }); });
185 |     element.append(buton);
186 |     container.append(element);
187 |   }
188 | }
189 |
190 | function valoareSortare(produs, cheie) {
```

Intrebări probabile și răspunsuri

De ce Math.max(1, ...)?

Răspuns: Pastrează o stare internă validă chiar dacă nu există rezultate; mesajul separat explică lista vidă.

De ce nu încarci o pagină nouă?

Răspuns: Această implementare face paginare client-side, permisă de cerință deoarece toate produsele sunt deja în DOM.

Ce face slice?

Răspuns: Returnează elementele dintre indexul de început inclusiv și cel final exclusiv.

6.B6 Fixare si doua tipuri de ascundere (0,50 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/produse` | VS Code: `resurse/js/produse.js`; `views/pagini/produse.ejs`

Ce cere exact: Fiecare card are trei butoane cu icon si tooltip: pastreaza la filtrare, ascunde pana la urmatoarea filtrare si ascunde in tab prin sessionStorage.

Demonstratia, in ordinea corecta

Fixeaza un produs si aplica un filtru care in mod normal l-ar exclude.

Ascunde temporar alt produs, apoi reaplica filtrarea si arata ca poate reveni.

Ascunde un produs pentru tab, reincarca si arata ca ramane ascuns.

Deschide pagina in alt tab si arata ca produsul exista acolo.

Cum functioneaza in proiect

Fixarea foloseste o clasa CSS; expresia corespunde OR fixat face exceptia explicita.

Ascunderea temporara foloseste dataset si se reseteaza la filtrare/sortare.

Ascunderea de tab salveaza id-ul in sessionStorage. Aceasta stocare este separata pentru fiecare tab si dispare cand tabul se inchide.

Delegarea evenimentului pune un singur listener pe lista si gaseste butonul/cardul cu closest.

Algoritmul pe care trebuie sa il intelegi

Intercepta clickul pe lista.

Gaseste cardul si actiunea apasata.

Pentru fixare comuta clasa si stilul butonului.

Pentru ascunderea simpla marcheaza dataset.

Pentru tab adauga id-ul in Set si serializeaza-l in sessionStorage.

Recalculeaza filtrarea sau paginarea.

Bucata-cheie: cele trei ramuri ale aceluiasi listener:

```

259 | produse.forEach((produs) => { produs.classList.remove("produs-card--fixat");
produs.dataset.ascunsTemporar = "false"; });
260 | aplicaFiltrare();
261 | }
262 |
263 | // Etapa 6 - Bonus 6: cele trei actiuni cerute pentru fiecare card.
264 | lista.addEventListener("click", function (eveniment) {
265 |   const card = eveniment.target.closest(".produs-card");
266 |   if (!card) return;
267 |   const fixare = eveniment.target.closest(".actiune-fixare");
268 |   const ascundere = eveniment.target.closest(".actiune-ascundere");
269 |   const sesiune = eveniment.target.closest(".actiune-sesiune");
270 |   if (fixare) {
271 |     card.classList.toggle("produs-card--fixat");
272 |     fixare.classList.toggle("btn-primary",
card.classList.contains("produs-card--fixat"));
273 |     fixare.classList.toggle("btn-outline-primary",
!card.classList.contains("produs-card--fixat"));
274 |     aplicaFiltrare();
275 |   } else if (ascundere) {
276 |     card.dataset.ascunsTemporar = "true";
277 |     actualizeazaPaginare();
278 |   } else if (sesiune) {
279 |     ascunseTab.add(card.dataset.produsId);
280 |     sessionStorage.setItem(cheieAscunseTab, JSON.stringify([...ascunseTab]));
281 |     aplicaFiltrare();
282 |   }
283 | });
284 |
285 | // Etapa 6 - Bonus 11: click pe zona neinteractiva a cardului deschide modalul.
286 | lista.addEventListener("click", function (eveniment) {
287 |   if (eveniment.target.closest("a, button, input, select, label")) return;
288 |   const card = eveniment.target.closest(".produs-card");
289 |   if (!card || !window.bootstrap) return;

```

Intrebari probabile si raspunsuri

localStorage si sessionStorage sunt identice?

Raspuns: Nu. localStorage persista intre sesiuni; sessionStorage apartine tabului curent.

De ce Set?

Raspuns: Set pastreaza id-uri unice si are verificare rapida prin has.

Ce este event delegation?

Raspuns: Ascultam pe parinte si identificam elementul copil care a declansat actiunea.

6.B7 Cautarea fara diacritice (0,15 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#)

Ce cere exact: Textul introdus fara diacritice trebuie sa gaseasca valori care contin diacritice si invers.

Demonstratia, in ordinea corecta

Cauta un termen existent mai intai cu diacritice, apoi fara.

Arata ca textul afisat nu este modificat.

In cod indica normalize NFD si expresia semnelor combinante.

Cum functioneaza in proiect

`toLocaleLowerCase('ro')` uniformizeaza majusculele in context romanesc.

`normalize('NFD')` descompune litera accentuata in litera de baza plus marcaj combinant.

Expresia `[\u0300-\u036f]` elimina marcatele; inlocuirile pentru `s` si `t` acopera forme Unicode care pot ramane.

Se normalizeaza copii folosite la comparatie, nu continutul vizibil.

Algoritmul pe care trebuie sa il intelegi

Converteste la litere mici.

Descompune Unicode.

Elimina diacriticele combinante.

Compara textele normalizate cu `includes`.

Bucata-cheie: functia mica reutilizata de mai multe filtre:

```
40 | function normalizeazaText(text) {
41 |   return text.toLocaleLowerCase("ro").normalize("NFD").replace(/[\u0300-\u036f]/g,
    |   "").replace(/ș/g, "s").replace(/ț/g, "t");
42 | }
43 |
44 | // Etapa 6 - punctul 10: validarea se executa inaintea operatiilor.
```

Intrebari probabile si raspunsuri

Ce este Unicode?

Raspuns: Standardul care atribuie coduri caracterelor; aceeași litera vizuala poate avea reprezentari compuse diferit.

Modifici numele produsului?

Raspuns: Nu. Transformarea este folosita exclusiv pentru comparatie.

6.B8 Sortarea dupa doua chei (0,35 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Utilizatorul alege doua chei dintre cel putin trei si sensul. Cheia secundara este folosita numai cand valorile primei chei sunt egale.

Demonstratia, in ordinea corecta

Alege pret ca prima cheie si nume ca a doua.

Sorteaza crescator si identifica primele rezultate.

Schimba prima cheie cu scor sau raport si sorteaza descrescator.

In cod indica primaComparatie si ramura de egalitate.

Cum functioneaza in proiect

valoareSortare intoarce Number pentru pret/scor/raport si text normalizat pentru nume/categorie.

comparaValori trateaza separat numerele si textele cu localeCompare.

Operatorul sens controleaza crescator/descrescator. Acelasi comparator este reutilizat.

Copierea [...produse] evita modificarea referintei colectiei de baza inaintea reordonarii DOM.

Algoritmul pe care trebuie sa il intelegi

Citeste cheile din selecturi.

Calculeaza prima comparatie.

Daca este nenula, intoarce rezultatul inmultit cu sens.

Daca este zero, calculeaza cheia a doua.

Muta nodurile sortate in lista.

Bucata-cheie: comparatorul cu departajare:

```

190 | function valoareSortare(produs, cheie) {
191 | if (cheie === "raport") return Number(produs.dataset.scor) /
    | Number(produs.dataset.pret);
192 | if (cheie === "pret" || cheie === "scor") return Number(produs.dataset[cheie]);
193 | return normalizeazaText(produs.dataset[cheie]);
194 | }
195 |
196 | function comparaValori(a, b) {
197 | if (typeof a === "number" && typeof b === "number") return a - b;
198 | return String(a).localeCompare(String(b), "ro", { numeric: true });
199 | }
200 |
201 | // Etapa 6 - punctul 8.2 si Bonus 8: sortare dupa doua chei alese.
202 | function sorteazaProduse(sens) {
203 | if (!valideazaInputuri(true)) return;
204 | const cheie1 = document.getElementById("cheie-sortare-1").value;
205 | const cheie2 = document.getElementById("cheie-sortare-2").value;
206 | const sortate = [...produse].sort(function (produsA, produsB) {
207 | const primaComparatie = comparaValori(valoareSortare(produsA, cheie1),
    | valoareSortare(produsB, cheie1));
208 | if (primaComparatie !== 0) return primaComparatie * sens;
209 | return comparaValori(valoareSortare(produsA, cheie2), valoareSortare(produsB,
    | cheie2)) * sens;
210 | });
211 | sortate.forEach((produs) => lista.append(produs));
212 | produse.forEach((produs) => { produs.dataset.ascunsTemporar = "false"; });
213 | actualizeazaPaginare();
214 | }
215 |
216 | // Etapa 6 - punctul 8.3: rezultatul este creat dinamic si sters dupa 2 secunde.

```

Intrebari probabile si raspunsuri

De ce ai nevoie de cheia a doua?

Raspuns: Oferă o ordine determinată când mai multe produse au aceeași valoare principală.

Ce face localeCompare?

Raspuns: Compară texte conform regulilor unei limbi și poate trata natural numerele incluse în siruri.

6.B9 Imagini multiple pentru produs (0,50 p)

Ce trebuie să deschizi: Browser: <http://localhost:8080/produs/1> | VS Code: module/imagini-produse.js; index.js; views/pagini/produs.ejs; resurse/imagini/produse-variante

Ce cere exact: Produsul are imagine principală și minimum două variante care pot fi schimbate printr-un carusel Bootstrap.

Demonstratia, in ordinea corecta

Deschide un produs și folosește controalele caruselului.

Indica faptul că toate cadrele aparțin aceluiași produs.

În folder arată fișierele -detaliu.webp și -complet.webp.

În modul arata Sharp și funcția `obtainImaginiProdus`.

Cum funcționează în proiect

La pornire pregătește Variante parcurge imaginile unice ale produselor. Sharp redimensionează și exportă WebP.

Variantele sunt derivate din imaginea produsului curent; nu se folosesc fotografii ale produselor similare.

`obtainImaginiProdus` construiește lista original plus două cai derivate. Ruta o pune în `produs.imagini`, iar EJS construiește `carousel-item`.

WebP reduce dimensiunea, iar Sharp automatizează generarea coerentă.

Algoritmul pe care trebuie să îl înțelegi

Determină sursa originală.

Construiește două destinații stabile.

Generează variantele dacă lipsesc sau trebuie actualizate.

Ruta cere lista imaginilor pentru produsul curent.

EJS marchează prima imagine activă și Bootstrap gestionează navigarea.

Bucata-cheie: generarea și returnarea celor trei cai:

```

1 | // Etapa 6 - Bonus 9: imagini multiple deduse din calea imaginii principale.
2 | const fs = require("fs");
3 | const path = require("path");
4 | const sharp = require("sharp");
5 |
6 | const folderProiect = path.join(__dirname, "..");
7 | const folderOriginale = path.join(folderProiect, "resurse", "imagini", "galerie",
  "originale");
8 | const folderVariante = path.join(folderProiect, "resurse", "imagini",
  "produse-variante");
9 | const extensiiAcceptate = new Set([".jpg", ".jpeg", ".png", ".webp"]);
10 |
11 | /**
12 |  * Construiește numele public al unei variante pornind din imaginea principală.
13 |  * @param {string} calePrincipala Calea salvată în tabela produse.
14 |  * @param {string} sufix Tipul cadrului generat.
15 |  * @returns {string} Calea HTTP a imaginii derivate.
16 |  */
17 | function caleVarianta(calePrincipala, sufix) {
18 |   const baza = path.parse(String(calePrincipala)).name;
19 |   return `./resurse/imagini/produse-variante/${baza}-${sufix}.webp`;
20 | }
21 |
22 | /**
23 |  * Etapa 6 - Bonus 9: generează pentru fiecare fotografie un cadru de detaliu și
24 |  * unul complet. Fisierul este refăcut numai dacă originalul este mai nou.
25 |  * @returns {Promise<void>} Promisiune rezolvată după generarea variantelor.
26 |  */
27 | async function pregatesteVariante() {
28 |   fs.mkdirSync(folderVariante, { recursive: true });
29 |   const fisiere = fs.readdirSync(folderOriginale).filter((nume) =>
30 |     extensiiAcceptate.has(path.extname(nume).toLowerCase()),
31 |   );
32 |
33 |   for (const nume of fisiere) {
34 |     const sursa = path.join(folderOriginale, nume);
35 |     const baza = path.parse(nume).name;
36 |     const variante = [
37 |       {
38 |         destinatie: path.join(folderVariante, `${baza}-detaliu.webp`),
39 |         transforma: (pipeline) => pipeline.resize(900, 650, { fit: "cover", position:
  "attention" }),
40 |       },
41 |       {
42 |         destinatie: path.join(folderVariante, `${baza}-complet.webp`),
43 |         transforma: (pipeline) => pipeline.resize(900, 650, { fit: "contain", background:
  "#f4f4f6" }),
44 |       },
45 |     ];
46 |
47 |     for (const varianta of variante) {
48 |       const trebuieGenerata =
49 |         !fs.existsSync(varianta.destinatie) ||
50 |         fs.statSync(sursa).mtimeMs > fs.statSync(varianta.destinatie).mtimeMs;

```

```
51 | if (trebuieGenerata) {
52 |
... fragment scurtat; functia completa ramane in fisier ...
```

Intrebari probabile si raspunsuri

De ce nu folosesti imaginile similare?

Raspuns: Cerinta cere imagini multiple ale aceluasi produs; acelea ar reprezenta alte produse.

Ce este Sharp?

Raspuns: O biblioteca Node pentru procesarea imaginilor: redimensionare, decupare si conversie.

De ce generezi la pornire?

Raspuns: Aplicatia se asigura automat ca variantele necesare exista inainte ca pagina sa le ceara.

6.B11 Modalul Bootstrap (0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [views/pagini/produse.ejs](#); [resurse/js/produse.js](#); [resurse/css/custom.css](#)

Ce cere exact: Clickul pe zona neinteractiva a cardului deschide un modal tematizat cu datele celui produs si posibilitate de inchidere.

Demonstratia, in ordinea corecta

Apasa pe suprafata libera a unui card, nu pe un buton.

Arata titlul, imaginea, descrierea, categoria, scorul si pretul.

Inchide cu X, cu butonul si prin click in exterior.

Apasa Pagina produsului si arata linkul dinamic.

Cum functioneaza in proiect

Listenerul este delegat listei. Guard-ul ignora linkurile, butoanele si inputurile pentru a nu deschide accidental modalul.

Datele sunt citite din dataset si introduse in elementele modalului.

`getOrCreateInstance` foloseste componenta JavaScript oficiala Bootstrap si reutilizeaza instanta.

Linkul este construit cu id-ul produsului curent.

Algoritmul pe care trebuie sa il intelegi

Gaseste cel mai apropiat card.

Opreste actiunea daca tinta este interactiva.

Completeaza titlul si continutul.

Seteaza href-ul produsului.

Obtine instanta Bootstrap si apeleaza `show`.

Bucata-cheie: popularea si deschiderea modalului:

```

291 | document.getElementById("modal-produs-continut").innerHTML = `<p>${card.dataset.descriere}</p><dl
class="row mb-0"><dt class="col-5">Categorie</dt><dd
class="col-7">${card.dataset.categorie}</dd><dt class="col-5">Performanță</dt><dd
class="col-7">${card.dataset.scor}/100</dd><dt class="col-5">Preț</dt><dd
class="col-7">${Number(card.dataset.pret).toFixed(2)} lei</dd></dl>`;
292 | document.getElementById("modal-produs-link").href =
`/produs/${card.dataset.produsId}`;
293 |
window.bootstrap.Modal.getOrCreateInstance(document.getElementById("modal-produs")).show();
294 | });
295 |
296 | // Etapa 6 - Bonus 20: comparatia persista cel mult o zi in localStorage.
297 | function citeșteComparatie() {
298 |   try {
299 |     const stare = JSON.parse(localStorage.getItem(cheieComparatie) || "null");
300 |     if (!stare || Date.now() - stare.actualizat > 24 * 60 * 60 * 1000) return [];
301 |     return Array.isArray(stare.produse) ? stare.produse.slice(0, 2) : [];
302 |   } catch { return []; }
303 | }
304 | function scrieComparatie(selectie) {

```

Intrebari probabile si raspunsuri

De ce nu scrii propriul modal de la zero?

Raspuns: Bonusul cere componenta Bootstrap; folosesc API-ul oficial pentru focus, fundal si inchidere.

Ce este `getOrCreateInstance`?

Raspuns: Returneaza instanta existenta pentru element sau creeaza una daca nu exista.

6.B13 Stergerea periodica a backupurilor CSS vechi (0,15 p)

Ce trebuie sa deschizi: Browser: Terminalul serverului si folderul backup/resurse/css | VS Code: module/curatare-backup.js; index.js; resurse/json/optiuni-server.json; backup/resurse/css

Ce cere exact: La un interval repetitiv, serverul verifica fisierele CSS din backup si le sterge numai pe cele mai vechi decat perioada T configurata.

Demonstratia, in ordinea corecta

In optiuni-server.json arata minutePastrareBackupCss si explica unitatea.

In folderul backup/resurse/css arata numele cu timestamp si datele fisierelor.

In index.js indica pornirea serviciului la initializarea serverului.

In modul indica apelul imediat, setInterval si conditia bazata pe mtimeMs.

Pentru o demonstratie controlata, foloseste un fisier de test vechi si o perioada mica; nu modifica fisiere importante.

Cum functioneaza in proiect

path.resolve construiește intern folderul exact backup/resurse/css, limitand operatia la zona prevazuta.

Configuratia este citita din JSON, transformata in Number si validata sa fie pozitiva.

limitaVechime este momentul curent minus T minute transformate in milisecunde.

Functia recursiva parcurge subfolderele; pentru fisiere accepta numai extensia .css.

fs.statSync(...).mtimeMs furnizeaza momentul ultimei modificari; fs.unlinkSync sterge numai daca acesta este sub limita.

setInterval repeta verificarea la 60 de secunde, iar unref permite procesului Node sa se inchida normal daca nu mai exista alta activitate.

Algoritmul pe care trebuie sa il intelegi

Valideaza intervalul de pastrare.

Calculeaza pragul de vechime.

Parcurge controlat folderul de backup.

Ignora directoarele neparcuse si fisierele care nu sunt CSS.

Compara mtimeMs cu pragul.

Sterge fisierul expirat, numara si raporteaza rezultatul.

Repeta verificarea cu setInterval.

Bucata-cheie: verificarea extensiei, compararea timpului si serviciul periodic:

```

1 | // Etapa 6 - Bonus 13: eliminarea periodica a backupurilor CSS expirate.
2 | const fs = require("fs");
3 | const path = require("path");
4 |
5 | /**
6 |  * Sterge numai fisierele CSS mai vechi decat perioada configurata.
7 |  * Folderul tinta este construit intern, nu primit din browser, astfel incat
8 |  * operatia ramane limitata la subfolderul backup/resurse/css.
9 |  * @param {object} optiuni Configuratia operatiei.
10 |  * @param {string} optiuni.folderBackup Folderul radacina al backupurilor.
11 |  * @param {number} optiuni.minutePastrare Perioada de pastrare in minute.
12 |  * @param {number} [optiuni.acum=Date.now()] Moment injectabil pentru testare.
13 |  * @returns {number} Numarul fisierelor eliminate.
14 |  */
15 | function curataBackupuriCssVechi({
16 |   folderBackup,
17 |   minutePastrare,
18 |   acum = Date.now(),
19 | }) {
20 |   const folderBackupCss = path.resolve(folderBackup, "resurse", "css");
21 |   const minuteValide = Number(minutePastrare);
22 |
23 |   if (!Number.isFinite(minuteValide) || minuteValide <= 0) {
24 |     throw new TypeError("minutePastrareBackupCss trebuie sa fie un numar pozitiv.");
25 |   }
26 |   if (!fs.existsSync(folderBackupCss)) return 0;
27 |
28 |   const limitaVechime = acum - minuteValide * 60 * 1000;
29 |   let numarSterse = 0;
30 |
31 |   function parcurge(caleFolder) {
32 |     for (const intrare of fs.readdirSync(caleFolder, { withFileTypes: true })) {
33 |       const caleIntrare = path.join(caleFolder, intrare.name);
34 |       if (intrare.isDirectory()) {
35 |         parcurge(caleIntrare);
36 |       } else if (
37 |         intrare.isFile() &&
38 |         path.extname(intrare.name).toLowerCase() === ".css" &&
39 |         fs.statSync(caleIntrare).mtimeMs < limitaVechime
40 |       ) {
41 |         fs.unlinkSync(caleIntrare);
42 |         numarSterse++;
43 |         console.log(`[Etapa 6 - Bonus 13] Backup expirat sters: ${caleIntrare}`);
44 |       }
45 |     }
46 |   }
47 |
48 |   parcurge(folderBackupCss);
49 |   return numarSterse;
50 | }
51 |
52 | /**
53 |  * Porneste verificarea imediata si apoi o repeta la fiecare minut.
54 |  * @param {object} optiuni Configuratia serviciului.

```

```

55 | * @param {string} optiuni.folderBackup Folderul radacina al backupurilor.
56 | * @param {string} optiuni.caleOptiuniServer Fisierul JSON cu perioada de pastrare.
57 | * @returns
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

Ce reprezinta mtimeMs?

Raspuns: Momentul ultimei modificari a fisierului, exprimat in milisecunde de la epoca Unix.

De ce filtrezi extensia .css?

Raspuns: Cerinta vizeaza backupurile compilarii SCSS in CSS; filtrul impiedica stergerea altor tipuri de fisiere.

De ce rulezi executa si inainte de setInterval?

Raspuns: Serverul curata imediat la pornire, fara sa astepte primul minut.

Ce face unref?

Raspuns: Timerul continua cat timp serverul ruleaza, dar singur nu tine procesul Node deschis.

Este sigura stergerea?

Raspuns: Tinta este construita intern si limitata la backup/resurse/css, iar conditia accepta numai fisiere CSS mai vechi decat pragul.

Atentie la prezentare: La prezentare nu sterge fisierele originale si nu seta o perioada mica fara sa folosesti un fisier de test. Explica mai intai limitele de siguranta ale functiei.

6.B14 Cel mai ieftin produs din fiecare categorie (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: index.js; views/pagini/produse.ejs

Ce cere exact: Pentru fiecare categorie se calculeaza pretul minim si produsele corespunzatoare primesc un marcaj clar.

Demonstratia, in ordinea corecta

Gaseste insigna Cel mai ieftin.

Compara pretul cu alt produs din aceeasi categorie.

In index.js arata Map-ul si cele doua treceri.

Cum functioneaza in proiect

Map asociaza fiecare categorie cu minimul curent. Prima parcurgere construiește minimele.

A doua parcurgere seteaza esteCelMaileftin comparand pretul produsului cu minimul categoriei.

Calculul se face pe server, iar EJS decide afisarea badge-ului.

Algoritmul pe care trebuie sa il intelegi

Porneste Map gol.

Pentru fiecare produs inlocuieste minimul daca pretul este mai mic.

Parcurge din nou produsele.

Marcheaza egalitatea cu minimul categoriei.

EJS afiseaza badge-ul conditionat.

Bucata-cheie: calculul minimului si proprietatea booleana:

```
834 | const pretMinimPeCategorie = new Map();
835 | produse.forEach(function (produs) {
836 |   const minimCurent = pretMinimPeCategorie.get(produs.categorie);
837 |   if (minimCurent === undefined || produs.pret < minimCurent) {
838 |     pretMinimPeCategorie.set(produs.categorie, produs.pret);
839 |   }
840 | });
841 | // Etapa 6 - Bonus 12d: pret redus calculat fara modificarea bazei.
842 | const ofertaCurenta = oferte.obtineOfertaCurenta();
843 | produse.forEach((produs) => {
844 |   if (ofertaCurenta && produs.categorie === ofertaCurenta.categorie) {
845 |     produs.oferta = ofertaCurenta;
846 |     produs.pretRedus = produs.pret * (1 - ofertaCurenta.reducere / 100);
847 |   }
848 | });
849 | const acum = Date.now();
850 | const intervalProdusNou = 120 * 24 * 60 * 60 * 1000;
851 | produse.forEach(function (produs) {
852 |   produs.esteCelMaiIeftin =
853 |     produs.pret === pretMinimPeCategorie.get(produs.categorie);
854 |   produs.esteNou =
855 |     acum - new Date(produs.data_adaugare + "T00:00:00").getTime() <=
856 |     intervalProdusNou;
857 | });
858 |
```

Intrebari probabile si raspunsuri

De ce doua treceri?

Raspuns: La prima descoperim minimul fiecărei categorii; abia apoi putem marca toate produsele care îl egalează.

Ce este Map?

Raspuns: O colecție de perechi cheie-valoare; aici cheia este categoria și valoarea este prețul minim.

6.B16 Produsele similare (0,30 p)

Ce trebuie să deschizi: Browser: <http://localhost:8080/produs/1> | VS Code: module/acces-produse.js; index.js; views/pagini/produs.ejs

Ce cere exact: Pagina produsului afișează recomandări selectate din bază după un criteriu clar și exclude produsul curent.

Demonstratia, in ordinea corecta

Deschide un produs și coboară la Produse similare.

Arata categoria produsului curent si a recomandarilor.

Apasa o recomandare si observa noul id din URL.

In SQL indica excluderea id-ului curent si LIMIT.

Cum functioneaza in proiect

Criteriul ales este categoria comuna. Query-ul parametrizat primeste categoria si id-ul.

Conditia id <> \$2 evita recomandarea paginii curente catre ea insasi.

LIMIT pastreaza zona scurta. EJS afiseaza informatii sumare si linkuri catre rutele produselor.

Algoritmul pe care trebuie sa il intelegi

Primeste obiectul produs curent.

Executa SELECT pentru aceeasi categorie.

Exclude id-ul curent.

Limiteaza numarul de randuri.

Transmite vectorul separat template-ului.

Bucata-cheie: interogarea produselor similare:

```

80 | const rezultat = await pool.query(
81 | `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
82 | pret::float8 AS pret, scor_performanta
83 | FROM produse
84 | WHERE id <> $1 AND categorie = $2::categorie_produș
85 | ORDER BY (subcategorie = $3) DESC, ABS(pret - $4), id
86 | LIMIT $5`,
87 | [produs.id, produs.categorie, produs.subcategorie, produs.pret, limita],
88 | );
89 | return rezultat.rows;
90 | }
91 |
92 | // Etapa 6 - Bonus 20: citirea stricta a celor doua produse cerute de comparatie.
93 | /** @param {number[]} ids Identificatori validati. @returns {Promise<object[]>}
    Produsele comparate. */
94 | async function obtineProduseDupaIduri(ids) {
95 | const rezultat = await pool.query(
96 | `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
97 | pret::float8 AS pret, scor_performanta, culoare::text,
98 | conectivitate, in_stoc
99 | FROM produse WHERE id = ANY($1::int[]) ORDER BY array_position($1::int[], id)`,
100 | [ids],
101 | );
102 | return rezultat.rows;
103 | }
104 |
105 | // Etapa 6 - Bonus 17: seturile, produsele lor si pretul redus sunt calculate in SQL.
106 | /** @param {number|null} [idProdus=null] Filtrare optionala dupa produs. @returns
    {Promise<object[]>} Seturi grupate. */
107 | async function obtineSeturi(idProdus = null) {
108 | const rezultat = await pool.query(
109 | `SELECT s.id,s.nume_set,s.descriere_set,
110 | json_agg(json_build_object('id',p.id,'nume',p.nume,'imagine',p.imagine,'pret',p.pret:
    :float8) ORDER BY p.id) AS produse,
111 | SUM(p.pret)::float8 AS pret_brut, COUNT(p.id)::int AS numar_produce
112 | FROM seturi s JOIN asociere_set a ON a.id_set=s.id JOIN produse p ON p.id=a.id_produș
113 | ${idProdus ? "WHERE s.id IN (SELECT id_set FROM asociere_set WHERE id_produș=$1)" :
    ""}
114 | GROUP BY s.id ORDER BY s.id`,
115 | idProdus ? [idProdus] : [],

```

Intrebari probabile si raspunsuri

De ce categoria este un criteriu bun?

Raspuns: Produsele au acelasi tip general si sunt relevante ca alternative; criteriul este usor de explicat si verificat.

De ce folosesti LIMIT?

Raspuns: Nu incarc inutil toata categoria si pastrez pagina usor de parcurs.

6.B17 Seturi de produse si reducerea calculata (1,00 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/seturi> si apoi pagina unui produs din set | VS Code: database/06-bonusuri-etapele6-8.sql; module/acces-produse.js; index.js; views/pagini/seturi.ejs; views/pagini/produs.ejs; views/fragmente/header.ejs

Ce cere exact: Proiectul are minimum cinci seturi. Relatia dintre seturi si produse este many-to-many, pagina /seturi afiseaza componentele si pretul redus, iar pagina fiecarui produs afiseaza seturile din care acesta face parte.

Demonstratia, in ordinea corecta

Din meniul Produse deschide Seturi PC Forge si numara cele minimum cinci seturi.

Pentru un set, indica numele, descrierea, produsele, pretul brut taiat, procentul si pretul redus.

Apasa numele sau imaginea unui produs si arata schimbarea URL-ului catre /produs/id.

Pe pagina produsului coboara la Seturi care includ produsul si arata ca apar numai seturile asociate.

In fisierul SQL arata cele doua tabele si asocierea prin cheile externe.

In acces-produse.js arata JOIN-urile, json_agg, SUM, COUNT si formula reducerii.

Cum functioneaza in proiect

Tabela seturi pastreaza id, nume_set si descriere_set.

Tabela asociere_set contine id_set si id_produs; ea rezolva relatia many-to-many fara liste de id-uri scrise intr-un singur camp.

JOIN reuneste setul, asocierile si produsele. GROUP BY produce un rand pentru fiecare set.

json_agg construiește vectorul produselor, SUM calculeaza pretul brut, iar COUNT determina numarul componentelor.

Reducerea este $\min(5, n) * 5\%$. Plafonarea la cinci inseamna maximum 25%, chiar daca setul contine mai multe produse.

Pretul redus este $\text{pret_brut} * (1 - \text{reducere}/100)$.

Parametrul optional idProdus adauga conditia WHERE cu \$1, astfel aceeași funcție poate alimenta atât /seturi, cat si pagina unui produs.

Algoritmul pe care trebuie sa il intelegi

Creeaza tabelele si cheile externe.

Populeaza minimum cinci seturi si asocierile lor.

Executa JOIN intre seturi, asociere_set si produse.

Grupeaza dupa set si agregă produsele.

Calculeaza suma, numarul produselor, procentul si pretul redus.

Ruta /seturi transmite toate seturile.

Ruta /produs/:id cere numai seturile care contin produsul curent.

EJS genereaza cardurile si linkurile produselor.

Bucata-cheie: interogarea many-to-many si calculul reducerii:

```

105 | // Etapa 6 - Bonus 17: seturile, produsele lor si pretul redus sunt calculate in SQL.
106 | /** @param {number|null} [idProdus=null] Filtrare optionala dupa produs. @returns
    | {Promise<object[]>} Seturi grupate. */
107 | async function obtineSeturi(idProdus = null) {
108 |   const rezultat = await pool.query(
109 |     `SELECT s.id,s.numa_set,s.descriere_set,
110 |     json_agg(json_build_object('id',p.id,'nume',p.nume,'imagine',p.imagine,'pret',p.pret:
    | :float8) ORDER BY p.id) AS produse,
111 |     SUM(p.pret)::float8 AS pret_brut, COUNT(p.id)::int AS numar_produce
112 |     FROM seturi s JOIN asociere_set a ON a.id_set=s.id JOIN produse p ON p.id=a.id_produs
113 |     ${idProdus ? "WHERE s.id IN (SELECT id_set FROM asociere_set WHERE id_produs=$1)" :
    | ""}
114 |     GROUP BY s.id ORDER BY s.id`,
115 |     idProdus ? [idProdus] : [],
116 |   );
117 |   return rezultat.rows.map((set) => ({ ...set, reducere: Math.min(5, set.numar_produce)
    | * 5, pret_redus: set.pret_brut * (1 - Math.min(5, set.numar_produce) * 0.05) }));
118 | }
119 |

```

Intrebari probabile si raspunsuri

Ce inseamna many-to-many?

Raspuns: Un set contine mai multe produse, iar acelasi produs poate apartine mai multor seturi.

De ce exista asociere_set?

Raspuns: O tabela intermediara reprezinta fiecare pereche set-produs si pastreaza baza normalizata.

Ce face json_agg?

Raspuns: Grupeaza randurile produselor intr-un vector JSON asociat unui singur set.

Cum calculezi reducerea pentru trei produse?

Raspuns: $\min(5,3)*5$ inseamna 15%; pretul redus este suma inmultita cu 0,85.

Dar pentru sapte produse?

Raspuns: $\min(5,7)$ este 5, deci reducerea ramane plafonata la 25%.

Cum eviti SQL injection?

Raspuns: Valoarea id-ului este trimisa separat prin parametrul \$1; nu este concatenata in textul interogarii.

De ce reutilizezi obtineSeturi?

Raspuns: Fara parametru returneaza toate seturile, iar cu idProdus returneaza numai seturile paginii curente.

Atentie la prezentare: Bonusul are patru subpuncte. Prezinta pe rand schema bazei, pagina tuturor seturilor, formula pretului si seturile din pagina produsului.

6.B18 Produsele noi (0,20 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/` si `/produse` | VS Code: `index.js`; `module/acces-orm.js`; `views/fragmente/produse-noi.ejs`; `views/pagini/produse.ejs`

Ce cere exact: Un produs este nou daca diferenta dintre data curenta si data adaugarii este cel mult intervalul T; primeste marcaj si apare intr-o sectiune ordonata invers cronologic.

Demonstratia, in ordinea corecta

Arata badge-ul Nou in `/produse`.

Pe prima pagina arata sectiunea cu cele mai noi produse.

Apasa un link din sectiune.

In cod indica `intervalProdusNou` si ordonarea descrescatoare.

Cum functioneaza in proiect

Data din baza este transformata intr-un obiect `Date`, iar diferenta in milisecunde este comparata cu 120 de zile.

`esteNou` este o proprietate booleana calculata pe server si consumata simplu de EJS.

Sectiunea principala foloseste ordonarea dupa `data_adaugare DESC`; implementarea ORM poate fi mentionata numai daca esti intrebati.

Algoritmul pe care trebuie sa il intelegi

Obtine data curenta.

Calculeaza diferenta pentru fiecare produs.

Seteaza `esteNou`.

Ordoneaza produsele noi descrescator dupa data.

EJS afiseaza badge-ul si linkul.

Bucata-cheie: regula de timp pentru marcajul Nou:

```
846 | produs.pretRedus = produs.pret * (1 - ofertaCurenta.reducere / 100);
847 | }
848 | });
849 | const acum = Date.now();
850 | const intervalProdusNou = 120 * 24 * 60 * 60 * 1000;
851 | produse.forEach(function (produs) {
852 |   produs.esteCelMaiIeftin =
853 |   produs.pret === pretMinimPeCategorie.get(produs.categorie);
854 |   produs.esteNou =
855 |   acum - new Date(produs.data_adaugare + "T00:00:00").getTime() <=
856 |   intervalProdusNou;
857 | });
858 |
```

Intrebari probabile si raspunsuri

Ce unitate are `Date.now`?

Raspuns: Milisecunde de la epoca Unix; de aceea si intervalul este convertit in milisecunde.

De ce data vine din baza?

Raspuns: Starea de produs nou trebuie sa depinda de data reala a inregistrarii, nu de HTML scris manual.

6.B19 Orarul disponibil pe orice pagina (0,20 p)

Ce trebuie sa deschizi: Browser: Orice pagina; butonul Program din interfata | VS Code: views/fragmente/orar.ejs; resurse/js/orar.js; views/fragmente/footer.ejs sau header.ejs

Ce cere exact: Un fragment EJS reutilizabil afiseaza toate zilele, marcheaza ziua curenta, spune Deschis/Inchis si poate fi deschis fara schimbarea paginii.

Demonstratia, in ordinea corecta

Deschide orarul de pe doua pagini diferite.

Indica marcarea zilei curente si mesajul Deschis/Inchis.

Inchide containerul si arata ca nu ai schimbat ruta.

In cod arata include-ul fragmentului si calculul zilei/orei.

Cum functioneaza in proiect

Fragmentul evita duplicarea aceluiasi tabel pe fiecare pagina.

JavaScript foloseste ziua si ora curenta pentru a selecta intervalul potrivit.

Afisarea este un strat peste pagina curenta; continutul poate fi inchis sau ascuns automat.

Algoritmul pe care trebuie sa il intelegi

Include fragmentul in structura comuna.

La deschidere citeste data curenta.

Gaseste randul zilei.

Compara ora cu intervalul.

Actualizeaza marcajul si starea, apoi afiseaza containerul.

Bucata-cheie: logica de afisare si calcul a starii programului:

```

1 | // Etapa 6 - Bonus 19: panou de program fara navigare, cu stare calculata dupa
2 | // ziua si ora curenta, evidentiarea zilei si inchidere automata.
3 | function initializeazaOrar() {
4 |   const buton = document.getElementById("btn-orar");
5 |   const panou = document.getElementById("panou-orar");
6 |   const inchide = document.getElementById("inchide-orar");
7 |   const stare = document.getElementById("stare-orar");
8 |   if (!buton || !panou || !inchide || !stare) return;
9 |
10 |   let temporizator;
11 |
12 |   function actualizeazaStare() {
13 |     const acum = new Date();
14 |     const rand = panou.querySelector(`[data-zi="${acum.getDay()}"]`);
15 |     panou.querySelectorAll("tbody tr").forEach((element) =>
16 |       element.classList.remove("zi-curenta"));
17 |     if (!rand) return;
18 |     rand.classList.add("zi-curenta");
19 |
20 |     const minuteAcum = acum.getHours() * 60 + acum.getMinutes();
21 |     const inMinute = (ora) => {
22 |       const [ore, minute] = ora.split(":").map(Number);
23 |       return ore * 60 + minute;
24 |     };
25 |     const esteDeschis = rand.dataset.inchis !== "true" &&
26 |       minuteAcum >= inMinute(rand.dataset.deschidere) &&
27 |       minuteAcum < inMinute(rand.dataset.inchidere);
28 |     stare.className = `alert mb-3 ${esteDeschis ? "alert-success" : "alert-secondary"}`;
29 |     stare.textContent = esteDeschis
30 |       ? `Suntem deschisi acum, pana la ${rand.dataset.inchidere}.`
31 |       : "Suntem inchisi acum. Consulta intervalele de mai jos.";
32 |   }
33 |
34 |   function ascundePanou() {
35 |     window.clearTimeout(temporizator);
36 |     panou.hidden = true;
37 |     buton.setAttribute("aria-expanded", "false");
38 |   }
39 |
40 |   function afiseazaPanou() {
41 |     actualizeazaStare();
42 |     panou.hidden = false;
43 |     buton.setAttribute("aria-expanded", "true");
44 |     window.clearTimeout(temporizator);
45 |     temporizator = window.setTimeout(ascundePanou, 12000);
46 |   }
47 |
48 |   buton.addEventListener("click", function () {
49 |     if (panou.hidden) afiseazaPanou();
50 |     else ascundePanou();
51 |   });
52 |   inchide.addEventListener("click", ascundePanou);
53 | }

```

```

54 |
55 | // Etapa 6 - Bonus 19: initializarea functioneaza si daca fisierul este incarcat
56 | // dupa evenimentul DOMContentLoaded (de exemplu din cache sau dintr-un fragment).

... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce fragment EJS?

Raspuns: Aceeasi structura este reutilizata pe toate paginile si se modifica intr-un singur loc.

Se face o pagina noua?

Raspuns: Nu. Cerinta cere vizualizarea fara schimbarea paginii, deci containerul apare peste continutul curent.

ETAPA 7 - selectia simpla: 1 punct

Ideea etapei: Inveti doar cele trei elemente usor de demonstrat: aparitia treptata a cardurilor, bannerul cookie si filtrele persistente.

Selectia PC Forge

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conexiuni	PCIe, USB-C, Ethernet
Adăugat	2 Mai 2025 (Sâmbătă)
Disponibil	Da



Platformă ATX pentru configurații AMD moderne, cu extensii rapide și traseu aerisit.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

☆
🔄
🕒
📄

899.90 lei
Detalii →

Placa de baza Forge Mini-ITX

Scor	76/100
Culoare	negru
Conexiuni	PCIe, Wi-Fi, USB-C
Adăugat	18 Aprilie 2025 (Sâmbătă)
Disponibil	Da



Placa compacta pentru sisteme de dimensiuni reduse, fara compromisuri de conectivitate.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

☆
🔄
🕒
📄

749.50 lei
Detalii →

Placa de baza Retro AT

Scor	25/100
Culoare	gri
Conexiuni	AT, PS/2
Adăugat	10 Noiembrie 2025 (Luni)
Disponibil	Nu



Model demonstrativ pentru prezentarea evolutiei arhitecturii calculatoarelor desktop.

CATEGORIE: COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

☆
🔄
🕒
📄

299.00 lei
Detalii →

1 2 3 4

Program

Cardurile Bootstrap sunt animate prin cooperarea dintre EJS, JavaScript si CSS.

7.1 Carduri Bootstrap afisate treptat (0,30 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/produse` | VS Code: `views/pagini/produse.ejs`; `resurse/js/produse.js`; `resurse/css/etapa7.css`

Ce cere exact: Cardurile tip Bootstrap apar la t , $2t$, $3t$ si asa mai departe, iar durata totala depaseste $3t$.

Demonstratia, in ordinea corecta

Reincarca `/produse` si nu interactiona in primele secunde.

Urmareste aparitia succesiva a cardurilor.

In cod arata $t=200$ si formula $(index+1)*t$.

In CSS arata starea initiala si clasa vizibila.

Cum functioneaza in proiect

EJS produce toate cardurile din prima. Ele nu sunt descarcate pe rand.

JavaScript adauga starea initiala si programeaza adaugarea clasei vizibile cu `setTimeout`.

CSS defineste `opacity/transform` si `transition`; JavaScript stabileste numai momentul.

`prefers-reduced-motion` dezactiveaza efectul pentru utilizatorii care solicita mai putina miscare.

Algoritmul pe care trebuie sa il intelegi

Colecteaza cardurile.

Pentru fiecare calculeaza intarzierea din index.

`setTimeout` inregistreaza callbackul.

Callbackul adauga clasa vizibila.

CSS interpoleaza intre stari.

Bucata-cheie: programarea neblocaanta a aparitiei:

```

20 | const cheieAscunseTab = "pc-forge-produse-ascunse-tab";
21 | const cheieComparatie = "pc-forge-comparatie";
22 | let paginaCurenta = 1;
23 | const ascunseTab = new Set(JSON.parse(sessionStorage.getItem(cheieAscunseTab) ||
    "[ ]"));
24 | const culoriValide = Array.from(document.querySelectorAll("#lista-culori
    option")).map((optiune) => optiune.value.toLowerCase());
25 |
26 | // Etapa 7 - cerinta 16 bootstrap_js: cardurile Bootstrap apar la t, 2t, 3t...
27 | // Pentru 20 de produse, ultimul apare la 4 secunde, deci animatia depaseste 3*t.
28 | function animeazaCarduriBootstrap() {
29 |   if (window.matchMedia("(prefers-reduced-motion: reduce)").matches) return;
30 |   const t = 200;
31 |   produse.forEach(function (produs, index) {
32 |     produs.classList.add("produs-card--animat");
33 |     window.setTimeout(function () {
34 |       produs.classList.add("produs-card--vizibil");
35 |     }, (index + 1) * t);
36 |   });
37 | }
38 |
39 | // Etapa 6 - Bonus 7: diacriticele sunt echivalente cu literele fara diacritice.

```

Intrebari probabile si raspunsuri

setTimeout blocheaza pagina?

Raspuns: Nu. Programeaza callbackul, iar firul poate continua celelalte operatii.

De ce nu pui delay doar in CSS?

Raspuns: Cerinta solicita realizarea in JavaScript cu setTimeout; CSS ramane responsabil de aspectul tranzitiei.

Cardurile nu exista pana apar?

Raspuns: Exista deja in DOM; initial sunt transparente/deplasate.

7.2 Bannerul animat si cookie-ul (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080> intr-o fereasta privata | VS Code: [views/fragmente/banner-cookies.ejs](#); [resurse/js/cookies.js](#); [resurse/css/etapa7.css](#)

Ce cere exact: Bannerul porneste mic si transparent din coltul stanga-jos, se animeaza cinci secunde si ramane. Acceptarea salveaza un cookie pentru sapte zile, iar bannerul dispare si nu reapare la refresh.

Demonstratia, in ordinea corecta

Deschide o fereasta privata si incarca pagina.

Urmareste animatia completa: dimensiune, opacitate si culoare.

Apasa OK si reincarca.

Arata cookie-ul `pc_forge_cookies_acceptate` si functiile de creare/stergere.

Cum functioneaza in proiect

`citesteCookie` cauta perechea `nume=valoare` in `document.cookie` si decodeaza valoarea.

seteazaCookie calculeaza data expirarii, adauga path=/ si SameSite=Lax.

La DOMContentLoaded, bannerul este pornit numai daca acceptarea lipseste. requestAnimationFrame asigura aplicarea starii initiale inainte de clasa animata.

Butonul OK seteaza cookie-ul si adauga starea de inchidere. Cookie-ul nu contine parola si nu este sesiunea de autentificare.

Algoritmul pe care trebuie sa il intelegi

Citeste cookie-ul de acceptare.

Daca exista, nu afisa bannerul.

Daca lipseste, elimina starea ascunsa si porneste animatia.

La OK scrie cookie-ul cu expirare sapte zile.

Ascunde bannerul; la refresh conditia initiala il exclude.

Bucata-cheie: utilitarele cookie si decizia de afisare:

```

1 | // Etapa 7 - animatie-banner si cookie-uri (cerinta individuala 0.3p).
2 | (function () {
3 | "use strict";
4 |
5 | /** Returneaza valoarea decodificata a cookie-ului cerut. */
6 | function citesteCookie(num) {
7 | const prefix = `${encodeURIComponent(num)}=`;
8 | const pereche = document.cookie.split("; ").find((element) =>
element.startsWith(prefix));
9 | return pereche ? decodeURIComponent(pereche.slice(prefix.length)) : null;
10 | }
11 |
12 | /** Salveaza un cookie pentru numarul indicat de zile. */
13 | function seteazaCookie(num, valoare, zile = 7) {
14 | const expirare = new Date(Date.now() + zile * 24 * 60 * 60 * 1000).toUTCString();
15 | document.cookie = `${encodeURIComponent(num)}=${encodeURIComponent(valoare)};
expires=${expirare}; path=/; SameSite=Lax`;
16 | }
17 |
18 | /** Etapa 7 - sterge cookie-ul primit ca parametru. */
19 | function deleteCookie(num) {
20 | document.cookie = `${encodeURIComponent(num)}=; expires=Thu, 01 Jan 1970 00:00:00 GMT;
path=/; SameSite=Lax`;
21 | }
22 |
23 | /** Etapa 7 - sterge toate cookie-urile accesibile paginii. */
24 | function deleteAllCookies() {
25 | document.cookie.split(";").forEach(function (pereche) {
26 | const num = decodeURIComponent(pereche.split("=")[0].trim());
27 | if (num) deleteCookie(num);
28 | });
29 | }
30 |
31 | // Functiile raman accesibile din consola pentru demonstratia ceruta.
32 | window.deleteCookie = deleteCookie;
33 | window.deleteAllCookies = deleteAllCookies;
34 |
35 | document.addEventListener("DOMContentLoaded", function () {
36 | const banner = document.getElementById("banner");
37 | const butonOk = document.getElementById("ok_cookies");
38 | const infoActivitate = document.getElementById("info-cookie-activitate");
39 | const cookieAcceptat = citesteCookie("pc_forge_cookies_acceptate");
40 |
41 | if (banner && !cookieAcceptat) {
42 | banner.hidden = false;
43 | requestAnimationFrame(function () {
44 | requestAnimationFrame(() => banner.classList.add("banner--vizibil"));
45 | });
46 | }
47 |
48 | if (butonOk) {
49 | butonOk.addEventListener("click", function () {
50 | seteazaCookie("pc_forge_cookies_acceptate", "da", 7);
51 | banner.classList.add("banner--inchis");

```

```
52 | window.setTimeout
... fragment scurtat; functia completa ramane in fisier ...
```

Intrebari probabile si raspunsuri

Cookie si localStorage sunt acelasi lucru?

Raspuns: Nu. Cookie-ul are expirare si este trimis in cererile compatibile; localStorage ramane in browser si este citit explicit de JavaScript.

Ce face SameSite=Lax?

Raspuns: Reduce trimiterea cookie-ului in anumite cereri pornite de alte site-uri, oferind o protectie de baza.

Poti sterge orice cookie cu deleteAllCookies?

Raspuns: Numai cookie-urile accesibile JavaScript-ului pentru domeniul si calea curenta; nu pe cele HttpOnly.

7.B3 Filtrele persistente (0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [views/pagini/produse.ejs](#); [resurse/js/produse.js](#)

Ce cere exact: Checkboxul Salveaza filtrele memoreaza valorile tuturor filtrelor in localStorage; la reintrare valorile sunt restaurate si filtrarea este reaplicata. Resetarea sterge si persistenta.

Demonstratia, in ordinea corecta

Bifeaza Salveaza filtrele.

Seteaza nume/scor/categorie si alte doua controale.

Reincarca pagina si arata ca inputurile si rezultatele revin.

Apasa Resetare, confirma, reincarca si arata ca starea nu mai revine.

Cum functioneaza in proiect

obțineStareFiltre creeaza un obiect serializabil. Valorile multiple devin vectori.

JSON.stringify transforma obiectul in text deoarece localStorage stocheaza numai siruri.

La incarcare JSON.parse reconstruieste obiectul si fiecare control este completat dupa tipul sau.

Nu se salveaza HTML-ul rezultat. Dupa restaurare se reaplica acelasi algoritm de filtrare.

try/catch elimina starea daca JSON-ul local este corupt.

Algoritmul pe care trebuie sa il intelegi

Colecteaza cele opt valori.

Serializeaza obiectul si il salveaza sub o cheie ce include URL-ul.

La incarcare citeste si parseaza cheia.

Completeaza text, range, radio, checkboxuri si selecturi.

Bifeaza optiunea de salvare si ruleaza aplicaFiltrare.

Resetarea debifeaza si removeItem sterge cheia.

Bucata-cheie: serializarea si restaurarea tuturor tipurilor de filtre:

```

77 | function obtineStareFiltre() {
78 |   return {
79 |     nume: inpNume.value,
80 |     scor: inpScor.value,
81 |     culoare: inpCuloare.value,
82 |     stoc: document.querySelector('input[name="stoc"]:checked').value,
83 |     categorii: valoriSelectate(".filtru-categorie"),
84 |     descriere: inpDescriere.value,
85 |     subcategorie: selectSubcategorie.value,
86 |     conectivitate: optiuniMultiple(selectConectivitate),
87 |   };
88 | }
89 |
90 | function salveazaStareFiltre() {
91 |   if (salveazaFiltre.checked) {
92 |     localStorage.setItem(cheieFiltrePersistente, JSON.stringify(obtineStareFiltre()));
93 |   }
94 | }
95 |
96 | function restaureazaStareFiltre() {
97 |   const stareSalvata = localStorage.getItem(cheieFiltrePersistente);
98 |   if (!stareSalvata) return;
99 |   try {
100 |     const stare = JSON.parse(stareSalvata);
101 |     inpNume.value = stare.nume || "";
102 |     inpScor.value = stare.scor || inpScor.min;
103 |     inpCuloare.value = stare.culoare || "";
104 |     inpDescriere.value = stare.descriere || "";
105 |     const radioStoc = document.querySelector(`input[name="stoc"][value="${stare.stoc} ||
"toate"]`);
106 |     if (radioStoc) radioStoc.checked = true;
107 |     document.querySelectorAll(".filtru-categorie").forEach((element) => {
108 |       element.checked = (stare.categorii || []).includes(element.value);
109 |     });
110 |     selectSubcategorie.value = stare.subcategorie || "";
111 |     Array.from(selectConectivitate.options).forEach((optiune) => {
112 |       optiune.selected = (stare.conectivitate || []).includes(optiune.value);
113 |     });
114 |     salveazaFiltre.checked = true;
115 |   } catch (eroare) {
116 |     localStorage.removeItem(cheieFiltrePersistente);
117 |   }
118 | }
119 |
120 | // Etapa 6 - cerintele 6 si 7: toate cele opt inputuri participa la filtrare.

```

Intrebari probabile si raspunsuri

De ce nu salvezi cardurile filtrate?

Raspuns: Rezultatul poate deveni inechit. Salvez intentia utilizatorului si recalculiez rezultatul din datele curente.

Ce se intampla daca textul din localStorage nu este JSON valid?

Raspuns: catch sterge cheia defecta si pagina continua cu valorile implicite.

Cand se sterg filtrele?

Raspuns: Numai cand utilizatorul debifeaza salvarea sau confirma Resetarea.

Repetitia finala - traseul extins de 25 de minute

Minutele 0-4: Etapa 5 - galerie statica, compilare SCSS si Bootstrap customizat.

Minutele 4-7: galeria animata, backupul si validarea galeriei.

Minutele 7-12: Etapa 6 obligatorie - baza, ruta produsului, opt filtre, sortare, calcul si resetare.

Minutele 12-17: paginare, fixare/ascundere, diacritice, modal, imagini si marcatele calculate.

Minutele 17-21: Bonusul 1, Bonusul 13 si Bonusul 17 - date dinamice, curatarea backupurilor si seturile.

Minutele 21-25: carduri animate, banner cookie si filtre persistente.

Daca timpul este foarte scurt: Arata mai intai Etapa 5 obligatorie, Etapa 6 obligatorie, paginarea, modalul si filtrele persistente. Acestea combina punctaj bun cu demonstratie clara.

Checklist inainte de prezentare

Docker Desktop si PostgreSQL sunt pornite.

npm start nu afiseaza eroare fatala.

/produse contine suficiente produse pentru doua pagini.

Fereastra privata este pregatita pentru banner.

Ai verificat ca galeria, imaginile produsului si orarul au fisierele necesare.

VS Code are deschise index.js, produse.js, produse.ejs, acces-produse.js, curatare-backup.js, cookies.js si custom.scss.

Nu este vizibil .env si nu folosesti un cont important pentru demonstratii.

Dictionarul minim pe care trebuie sa il stapanesti

EJS: motor de template care combina HTML-ul cu datele serverului.

SCSS/Sass: sursa CSS extinsa cu variabile, nesting si bucle, compilata in CSS.

DOM: reprezentarea obiectuala a paginii pe care JavaScript o poate modifica.

dataset: accesul JavaScript la attributele HTML data-.*.

Promise/await: reprezentarea si asteptarea unui rezultat asincron.

callback: functie transmisa pentru a fi apelata ulterior.

localStorage: stocare persistenta in browser, netrimisa automat serverului.

sessionStorage: stocare separata pentru tab, pierduta la inchiderea lui.

cookie: valoare cu expirare si cale, trimisa in cererile corespunzatoare.

media query: regula CSS activata de caracteristicile ecranului sau preferintele utilizatorului.

query parametrizat: SQL in care valorile sunt trimise separat prin \$1, \$2.